2023
אילת לוי

סיכום מושגים חשובים לקראת המבחן

מבוא

* מלמעלה למטה – מספקת למשתמש ממשק המאפשר שימוש בחומרה.
* מלמטה למעלה – מנהלת את משאבי המערכת.

* System call – ממשק של קריאות מערכת דרכו המשתמש יכול לבקש ממערכת ההפעלה שירותים שדורשים הרשאה מיוחדת. כל קריאה מעבירה את השליטה מהמשתמש אל מערכת ההפעלה באופן זמני.
TRAP – מעבירה את המעבד למצב מיוחס וגורמת למעבר השליטה למערכת ההפעלה.

מבנים של מערכות הפעלה-

* מונוליתיות
* מרובדות
* מכונה מדומה
* exokernal
* micro-kernal
* שרת– לקוח

תהליכים

סיום תהליכים -

* סיום רגיל
* סיום בעקבות שגיאה
} מתוכנן

* סיום בעקבות שגיאה פטאלית
* סיום בעקבות הריגת תהליך
} לא מתוכנן

$$\frac{\text{גודל בלוק}}{\text{כתובת בלוק}} = \text{מס' רשומות שיש בבלוק יחיד.}$$

$$\frac{\text{גודל דף}}{\text{גודל מילה}} = \text{גודל ה offset}$$

מאפיינים של תהליכונים

* מצב התהליכון
* מצב הרגיסטרים של המעבד
* מחסנית נפרדת עבור כל תהליכון
* אפשרות גישה לזכרון ולמשאבים המשותפים בתוך תהליך.

* לתהליכונים של אותו תהליך גישה למרחב הזכרון המשותף. שינוי משתנה גלובלי משפיע על כולם.

יתרונות לשימוש בתהליכונים

* יצירת תהליך חדש מצריכה משאבים רבים יחסית לדרישות של תהליכון.
* זמן ההשמדה של תהליכים קצר יותר.
* החלפת תהליכונים מצריכה שמירת תוכן לשחזור עתידי וכמובן מהיר יותר לשמור תוכן של תהליכון מאשר של תהליך.
* תקשורת בין תהליכים דורשת מעורבות ה kernel ואילו בין תהליכונים אין צורך.

* כדאי לשקול שימוש בתהליכים במקרה של אפליקציה שנותנת מענה לאנשים רב של אירועים א-סינכרוניים.
לכל מאורע כזה ניתן ליצור תהליכון שיתן מענה מתאים.

מימוש תהליכונים ברמת המשתמש

יתרונות

* החלפת תהליכונים ברמת המשתמש אינה דורשת מעבר למצב גרעיני.
* שימוש תהליכונים בספרייה ברמת המשתמש מאפשר את התאמת אלגוריתם התזמון לצורך של כל יישום נפרד.
* ניתן לתמוך בתהליכונים גם כאשר גרעין מערכת ההפעלה לא תומך בכך.

חסרונות

* אם אחד התהליכונים בתהליך מבצע קריאה שגורמת לו לעבור למצב חסום אז כל התהליך עובר למצב חסום למרות שייתכנו תהליכונים שהם במצב מוכן. זה מוריד את רמת המקביליות.
* מאפשר לתהליכונים להפעיל את המעבד אחד מהם וזה פוגע ברמה התכנונית של השימוש בתהליכונים כמו ביצוע נפרדים.

מימוש תהליכונים בגרעין המערכת

מתגבר על החסרונות של השיטה הקודמת.
מסרון משמעותי - בלבוש הזמן כרוך במעבר ממצב משתמש למצב גרעיני.

התנאים הדרושים לפתרון סביר לקטע הקריטי

* שני תהליכים לא יימצאו בו זמנית בתוך קטע קריטי. מניעה הדדית
* הפתרון לא יסתמך על השערות הקשורות למהירות או מספר מעבדים.
* תהליך מחוץ לקטע הקריטי לא ימנע מתהליך אחר להיכנס לקטע הקריטי. התקדמות.
* תהליך לא ימתין לנצח כדי להיכנס לקטע הקריטי. הוגנות

מניעה הדדית הכוללת המתנה פעילה

*** מניעת הפסיקות** – לא סביר. **יתרון** – המעבד לא עובר לתהליך אחר לכן אין חשש שתהליך אחר ימצא בקטע הקריטי.

חסרונות –
- תהליכי משתמש יכולים למנוע מתהליכים אחרים לקבל זמן מעבד עקב שגיאת תכנות. פגיעה בטיחות.
- ירידה ברמת המקביליות
- הרשימה רלוונטית למעבד אחד לכן ישנו צורך לחסום בכל המעבדים וזה גם כן מביא לירידה ברמת המקביליות.

– שיטה זו שימושית לגרעין ונדרשת לעיתים אך לא סביר להשתמש בה עבור תהליכי משתמש.

*** משתני נעילה** –
יש משתנה משותף lock שמאותחל ל-0. מאפסים ובודקים אם רוצים להיכנס לקטע הקריטי.
לא פותר את בעיית המירוץ – **לא מתקיים התנאי של מניעה הדדית (תנאי 1).**

*** פתרון התור** –
משתנה משותף חדש turn שמאותחל ל-0 ועוזר מספק את מס' התהליך שתורו להיכנס לקטע הקריטי.
מקיים מניעה הדדית (שני תהליכים לעולם לא יהיו ביחד בקטע קריטי).
המתנה פעילה – תהליך מקבל את המעבד ומנצל את מלוא הזמן המוקצב לו, אך איננו מתקדם כלל.
הרעבה – תהליך ממתין לאיטו מסיים זמן רב ולא יכול להתקדם עד אז.
– הפתרון מפר את תנאי 3 – תהליך שנמצא מחוץ לקטע קריטי יכול למנוע מתהליך אחר להיכנס לקטע הקריטי שלו.

*** דקר ופיטרסון** –
שילוב של פתרון התור בתוספת משתנה נעילה.
פיטרסון –
מערך interested[2]; bool int turn.
תהליכים מצהירים על רצונם להיכנס לקטע הקריטי ע"י שינוי תא במערך interested.
אם רק אחד רוצה להיכנס ברור שהוא נכנס. **תנאי 3 מתקיים.**
אם שניהם רוצים להיכנס, הראשון שעדכן את turn יכנס. **לכן תנאי 1 מתקיים.**
תהליך שרוצה להיכנס לקטע הקריטי נכנס אלא אם כן תהליך אחר כבר **נכנס לכן תנאי 4 מתקיים.**
כמובן שגם **תנאי 2 מתקיים**, **לכן כל התנאים מתקיימים וזה פתרון טוב.**

*** TSL – Test Set Lock**
TSL היא פעולה אטומית.
אם lock=0 משנה אותה ל-1 ומחזירה 0
אם lock≠0 מחזירה את ערכה.
פתרון זה משתמש במשתנה משותף lock עליו מבצעים את הפעולה TSL.
אחרי הקטע הקריטי מאפסים lock כדי לשחרר את המנעול.
*** כשמדובר בשני תהליכים מתקיימים 4 התנאים וזה פתרון טוב.**
אם יש כמה תהליכים הפתרון יכול לגרום להרעבה כי פתרון זה לא מסדר את אופן הכניסה של תהליכים לקטע הקריטי.

מניעה הדדית ללא המתנה פעילה

נרדה פתרון העונה על התנאים הבאים:

* מקיים את 4 התנאים של פתרון סביר לכמה תהליכים
* אינו גורם להמתנה פעילה
* משפר את אופן הטיפול בתהליכים שרצו להיכנס לקטע הקריטי ולא יכלו כי הקטע הקריטי היה תפוס.
* אינו מדגיש ניהול מצבים הדורשים היפוך עדיפויות.

בפתרונות הבאים נשתמש בקריאות מערכת SLEEP, WAKEUP. תהליך יכול להשהות את עצמו ולהתעורר. תהליכים שמבצעים קריאות אלה עוברים למצב חסום. כך מקבלים המתנה רדומה שאינה מעסיקה את המעבד. מערכת ההפעלה יכולה לנהל תור של תהליכים רדומים ורק להסדיר את סדר כניסתם לקטע הקריטי.

* סמפורים

מונה S של מס' קריאות ל WAKEUP שעתידות להתבצע. S הוא סמפור - מנגנון המסופק ע"י מערכת ההפעלה או שפת תכנות.
פעולות down(S), up(S). אטומיות!
אם מאתחלים את S לערך קיומי לעולם לא יקבל ערך שלילי. ערכו מסמן כמה תהליכים יכולים להיות בו במקביל באותו סמפור.
- משמש down לפני כניסה לקטע קריטי, up ביציאה ממנו.
* פתרון בעיית היצרן-צרכן עם סמפורים מקיים את 4 התנאים לפתרון סביר.

* מנעולים Mutex

מנעול הוא משתנה במערך מבנן משותף המשמש לניהול הגישה לקטע קריטי.
- דומה לסמפור בינארי - יש לו 2 מצבים - נעול ולא נעול.
- בשונה מסמפור בינארי הוא יעיל יותר מכיוון שאין צורך מעבר בין רמת המשתמש לרמת הגרעין.
משתמשים ב TSL כדי לממש אותו.

* Futex

משלב יתרון של גישת המתנה פעילה וללא המתנה פעילה. בהליכון מנסה לתפוס את המנעול עם TSL.

* מבני פיקוח Monitors

- שימוש בסמפורים כבם על פני התכניות הופכת אותה לקשה לניפוי שגיאות.
מבנה פיקוח הוא ספרייה של שגרות משתנים ומבני נתונים המוגדרים יחדיו במבנה של שפת תכנות והם כי מבטיחה שתהליכים הקוראים לשגרות המוגדרות במבנה פיקוח ממומשים מניעה הדדית.
הקומפיילר מופקד על מימוש מניעה הדדית לכן יש פחות סיכוי לטעויות שיובילו לקיפאון.
מוגדרים משתנים מיוחדים Signal(c) מעירה תהליך שהיה חסום. wait(c) גורמת להשהיית תהליך כל עוד תנאי עוד לא התקיים.
מבנה פיקוח מבטיח שלאחר הקריאה ל wait רק אחד מהתהליכים שהיה חסום על קטע קריטי יוכל להיכנס אליו.

* העברת הודעות Message Passing

send ו- receive.

מיעון ישיר rendezvous

מיעון עקיף mailbox משתמש בתחנות ביניים.

* מחסומים Barriers

נשמרים להשהיות קבוצה שלמה של תהליכים עד אשר יתקיים תנאי כלשהו.

תזמון תהליכים Scheduling

נתמקד במעברים ממצב ready למצב running.

קריטריונים להערכה של אלגוריתמי תזמון לזמן קצר

* בכל סוגי המערכות – הוגנות, אכיפת מדיניות, איזון.
* במערכות אצווה – קצב העבודה, זמן השהייה, ניצולת CPU.
* במערכות אינטראקטיביות – זמן תגובה, יחסיות.
* מערכות זמן אמת – עמידה בזמנים, ניתנות לחיזוי.

תזמון במערכות אצווה

* First Come First Served – FCFS. FIFO של תהליכים במצב ready.
  - יתרון – פשוט למימוש, הוגן.
  - חסרון – זמן השהייה ארוך.

* Shortest Job First – SJF בתנאי שזמני הריצה של התהליכים ידועים מראש נותנים CPU לתהליך עם זמן ריצה קצר ביותר עד לסיום ללא הפקעת המעבד.
  - יתרון – זמן שהייה מזערי
  - חסרון – צריך לדעת מראש זמן ריצה ידוע של כל תהליך.

* Shortest Remaining Time Next – SRTN כמו SJF רק שהוא כן מפקיע את המעבד.
  - יתרון – זמן השהייה מזערי
  - חסרון – דורשת ידיעה מוקדמת של זמן הריצה של כל תהליך.

תזמון במערכות אינטראקטיביות

* Round Robin – RR - מחלקים זמן ריצה קבוע לכל תהליך quantum. התהליכים מנוהלים בתור מעגלי.
  קביעת quantum קצר מדי – תקורה גבוהה של זמן המתבזבז על החלפת תהליכים.
  ארוך מדי – זמן תגובה ארוך, פגיעה בתהליכים אינטראקטיביים.

* תזמון עם עדיפויות (קדימות) – התהליכים המוכנים מדורגים לפי עדיפויות. תהליך קבל עדיפות גבוהה רץ כל עוד עדיפותו גבוהה מעדיפותם של שאר התהליכים המוכנים לריצה.
  עדיפות קבועה – יעיל במקרה של תהליכים שדורשים זמן תגובה מהיר מאוד.
  עדיפות דינאמית – עדיפותו של תהליך תקבע לפי החלק היחסי של quantum שנוצל על ידיו. ככל שהחלק המנוצל גדול יותר כך עדיפותו של התהליך קטנה יותר.
  - מספק זמן תגובה טוב לתהליכים הנמצאים רוב הזמן במצב רדום.
  - חסרון – קשה להתאים שיטה כזו לקבוצת הרחבה של סוגי השירות שתהליכים דורשים.

* ריבוי תורים – תורים לפי סוג השירות שהתהליכים מבקשים. לכל תור יש עדיפות (תהליכים באותו תור בעלי עדיפות שווה), וסוג השירות שהתהליכים הנמצאים בתור נגזר ממנה. תהליכים יכולים לעבור בין תורים בהתאם מההעדפות המתזמן.

* Shortest Process Next - SPN

ניבוי הזמן הדרוש לפעולה הבאה בהסתמך על ההיסטוריה של התהליך. שיטת ניבוי המבוססת על aging

$T(i)$ הזמן שנדרש בפועל לבצע את הפעולה ה-i-ית

$E(i)$ הזמן המשוער לבצע את הפעולה ה-i-ית

$E(1)$ נתון תמיד

$$E(i+1) = (1-a)T(i) + aE(i) \qquad 0<a<1$$

-שיטה זו יכולה לגרום להרעבה של תהליכים שהפעולה שלהם ארוכה מהממוצע של ביצוע פעולה במערכת.
ריבוי תורים יכול לעזור להתגבר על הבעיה.

* Lottery Scheduling

יתרונות - הפחתת תקורה של תיעוד המידות המוקצות
קל לתת עדיפות באמצעות התחשבות של תהליך.
תהליכים שמשתפים פעולה יכולים להעביר כרטיסים זה לזה וכך מאפשר פתרונות נוספים לבעיות כמו יצרן-צרכן.

* Fair Share Scheduling

יש לקחת בחשבון את השתייכותם של תהליכים לקבוצה מסויימת.

תזמון במערכת זמן אמת

יכול להיות סטטי או דינמי
משימות מחזוריות מתבצעות עם תקופת מחזור מסוימת.

$C(i)$ מס' יחידות זמן התיפוס ב-CPU הדרושות למשימה מחזורית i

$P(i)$ משך זמן המחזור של המשימה ה-i-ית.

הקריטריון לאפשרות לתזמן m משימות $1 \le i \le m$:

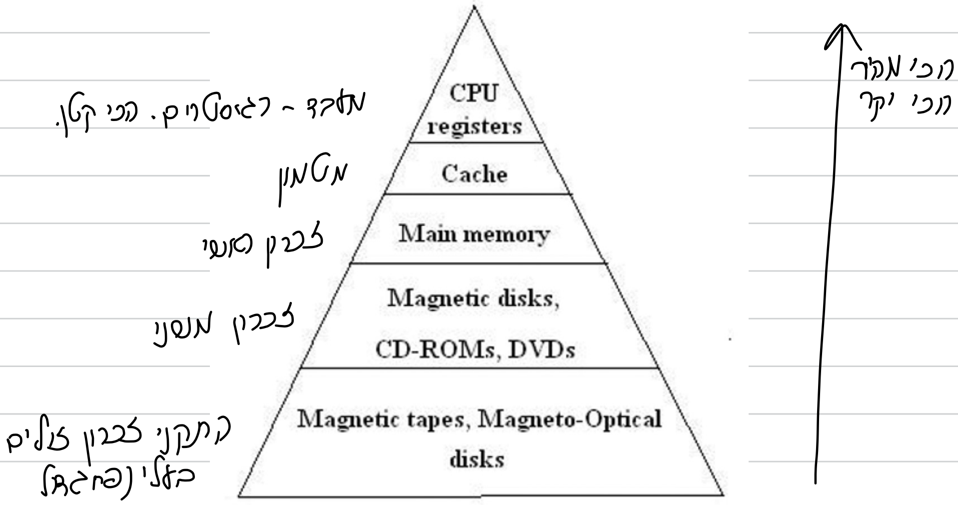
$$\sum_{i=1}^{m} \frac{C(i)}{P(i)} = \frac{C(1)}{P(1)} + \dots + \frac{C(m)}{P(m)} \le 1$$

מנגנון מול מדיניות

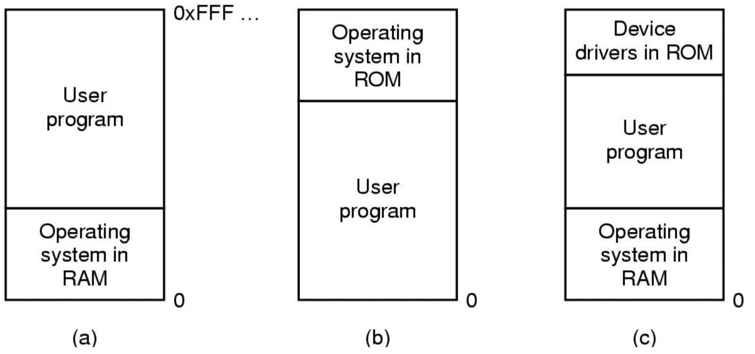
מערכת ההפעלה מספקת מנגנונים, בכך השימוש נקבע ע"י המשתמשים.
למשל ב-RR המנגנון מאפשר להריץ תהליכים בתוך quantum q, המדיניות נקבעת לפי האורך שלו.

ניהול זכרון

היררכית זכרון



שיטות לניהול זכרון ישיר



ניידות תכניות במרחב זכרון

* תרגום מרחב הכתובות הוירטואלי למרחב הכתובות הפיזי מתבצע בחומרה (צריך להיות מהיר).
בחומרה שעושה זאת יש שני אוגרים מיוחדים: limit - מען המחיצה של התהליך, base - גודל המחיצה.
בזמן התרגום בודקים אם מנסים לגשת לאוגר שחורג מה- limit. מעיד על ניסיון גישה בלתי חוקי. גורם ל protection fault
אם הכתובת חוקית, מוסיפים לכתובת הוירטואלית את הערך של base והתוצאה היא הכתובת הפיזית.

הרכיב האחראי לתרגום כתובת וירטואלית לפיזית נקרא - MMU - Memory Management Unit.

החלפה בזכרון

תהליך מוחזק בזכרון הראשי כשהוא רץ, שומרים אותו בדיסק כשהוא עובר למצב חסום ושוב מעתיקים אותו כשהוא מוכן לרוץ.

ריסוק חיצוני - תופעה בהקצאה של זכרון שמהותה היווצרות שטחי זכרון פנויים בין השטחים שמוקצים לתהליכים, כאשר השטחים הפנויים קטנים מכדי להכיל תהליך נוסף ולכן אינם מנוצלים.
צימצום - כדי להתגבר על ריסוק חיצוני, העתקה של תהליכים שמפוזרים בשטחים שונים של הזכרון כדי שימוקמו בצורה קומפקטית בצד אחד, כך הזכרון מנוצל בצורה טובה יותר. פעולה זו יקרה מאוד. בד"כ לא משתמשים.
ריסוק פנימי - תופעה בהקצאה של זכרון הנגרמת כתוצאה מהקצאת זכרון עודפת לתהליך (מקבל מחיצה גדולה יותר ממה שהוא צריך). תופעה זו קיימת כאשר גודל המחיצות קבוע או כאשר גודל המחיצה חייב להיות כפולה של יחידת הקצאה מינימלית.

ניהול זכרון פנוי

* הקצאת שטחי זכרון פנויים באמצעות מפת סיביות - bits-map
מחלקים את הזכרון למחיצות בגודל קבוע. 0 קטע פנוי, 1 קטע תפוס.
הקצאה של זכרון מתבצעת ע"י חיפוש של סדרת אפסים רציפה במפת הסיביות כך שהקטעים המתאימים יספיקו להכיל את התהליך.
קטעים גדולים יותר - בממוצע ½ קטע האחרון. (חסרון)
מפת סיביות קטנה יותר. (יתרון)
} הפוך בקטעים קטנים
יתרון - תקורה ניהולית קבועה לאחר שנקבע גודל מחיצה.
חסרון - חיפוש איטי במפת הסיביות.

*הקצאת שטחי זכרון פנויים באמצעות רשימות מקושרות Linked lists

- First Fit - FF - סורקים את הרשימה המקושרת מההתחלה עד שמוצאים שטח גדול מספיק להכיל את התהליך. מפצלים את שטח זה - מוקצה לתהליך והפנוי שאין בו צורך לתהליך הופך להיות שטח פנוי להקצאה באיטרציה הבאה.
*יתרונות - מימוש זה מהיר כי הסריקה נעצרת ברגע שמוצאים שטח פנוי מתאים.
- מימוש זה פשוט למימוש.
* חסרונות - ריסוק חיצוני - שטח פנוי בתחילת הזכרון ואח"כ שטחים מוקצים.
הכי טוב מבין כולם.
- Next Fit - NF - שיפור של FF, במקום לסרוק כל פעם מתחילת הרשימה מבצעים סריקה מעגלית שמתחילה מהמקום האחרון בו הקצנו בפעם הקודמת.
- יתרונות - גם כן מהיר כמו FF
- חסרונות - ריסוק חיצוני. תבניות הריסוק שלו פחות טובות. נוטה לפצל את השטח הפנוי בסוף הזכרון וצריך יותר פעולות צירוף.
- Best Fit - BF - סורקים את כל הרשימה ומוצאים את השטח הפנוי הקטן ביותר שמספיק להכיל את התהליך.
- חסרונות - הכי גרוע מבין כולם. ריסוק חיצוני - מייצר הרבה שטחים פנויים שקשה לנצל בגלל הגודל שלהם. צריך הרבה פעולות צירוף.
- Worst Fit - WF - סורקים את כל הרשימה ומוצאים את השטח הפנוי הגדול ביותר לתהליך.
- חסרונות - יוצר ריסוק פנימי גדול.
- Quick Fit - QF - שכלול הרעיון של רשימה מקושרת ע"י החזקה של מס' רשימות של שטחים פנויים. כל רשימה טווח אחר של גדלים.
- יתרון - מציאת שטח פנוי מהירה מאוד
- חסרון - בעת שחרור שטח במקרה הכי גרוע צריך לעבור על כל הרשימות כדי לאחד שטחים וזה לוקח הרבה זמן.

זכרון מדומה

זכרון וירטואלי (מדומה) - שיטה בה מאחסנים חלק מהתהליך בזכרון - רק החלק שחיוני לעבודה ברגע נתון נמצא בזכרון הראשי וחלקים אחרים יכולים להיות מאוחסנים בדיסק. שיטה זו מאפשרת הרצה של תהליכים שגודלם עולה על גודל הפיזי במערכת.

למדנו 3 שיטות למימוש זכרון וירטואלי - דפדוף, סגמנטציה, שילוב של דפדוף עם סגמנטציה.

דפדוף

- מרחב הכתובות של תהליך מחולק לדפים בגודל קבוע.
- הזכרון הראשי מחולק למסגרות בגודל הזהה לגודל הדפים של התהליכים.
- דפים יכולים להיות ממופים למסגרות פיזיות בזכרון הראשי או להימצא בזכרון המשני.
- אם מתבצעת פניה ל VA שהדף המכיל אותה נמצא בזכרון המשני פניה זו תגרום לפסיקה page fault interrupt. פסיקה זו גורמת למערכת ההפעלה לתזמן בקשה להביא את הדף החסר לזכרון הראשי. כשהדף מגיע לזכרון הראשי מעדכנים את טבלת הדפים שהדף נמצא בזכרון הראשי. התכנית תמשיך לבצע את ההוראות שניתנו לפני הפסיקה.
- מס' דף משמש כאינדקס שורה בטבלת הדפים.

VA

מס' הדף	offset
כמות הדפים	גודל דף

PA

מס' המסגרת	offset
כמות המסגרות	גודל המסגרת

מבנה של שורה בטבלת הדפים -

Chaching disabled bit, modified bit, reference bit, protection field, present/absent bit, page frame number

טבלת הדפים בבריאה הבסיסית גדולה מדי, מספיק להחזיק רק חלקים של טבלת הדפים המתייחסים לדפים בשימוש.

הגברת מהירות פניות כתובות

Translation Lookaside Buffer - TLB

- בTLB מוחזקת קבוצה קטנה של שורות מטבלת דפים.
- אין referenced bit. הוצאת דפים מהזכרון הראשי זה אחריות של מערכת ההפעלה ולא של החומרה.
- נפח מוגבל. אינה מכילה מידע על דפים לשימושו של האלגוריתם להחלפת דפים.
- שדה valid מציין אם השורה פנויה או שהיא מאחסנת מידע לגבי דף.
- מתבצע חיפוש אסוציאטיבי - על כל השורות בבת אחת.

ניתן לפנות שורות של טבלת ה TLB לפי LRU - השורה המוחלפת תהיה זו שלא השתמשו בה במשך זמן הכי ארוך.

תהליך כללי:

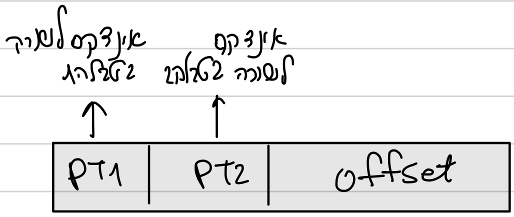
- מנגנון תרגום המענים MMU בודק אם הדף נמצא ב TLB. אם כן מבצע תרגום של VA.
אם לא, הולך לחפש בטבלת הדפים בזכרון הראשי!
אם הדף נמצא בטבלה זו - MMU מעדכן את TLB ומבצע תרגום של VA.
אם הדף לא נמצא בטבלה זו - יש page fault, מערכת ההפעלה מביאה את הדף לזכרון הראשי!
לאחר מכן מעדכנים את טבלת הדפים ואז גם את ה TLB ואז מבצעים תרגום של VA.

מיפוי בלוקים של זכרון ראשי לשורות של זכרון מטמון

- מיפוי ישיר - כל בלוק של זכרון ראשי יכול להיות ממופה אך ורק לשורה מסוימת של זכרון מטמון.
- שיטת מיפוי דו אסוציאטיבית - כל בלוק של זכרון ראשי יכול להיות ממופה לאחת משתי שורות של זכרון מטמון.
שיטה זו דורשת חיפוש במקביל בשתי שורות.

זכרון מטמון יכול להיות במס' רמות למשל L_1, L_2
* Exclusive - המידע ברמה 1 אינו נמצא ברמה השנייה.
* Inclusive - הם יכולים להכיל את אותו המידע.

טבלת דפים שכבתית -



מס' השכבות נקבע כך שטבלאות דפים של כל רמה יכנסו לדף בודד.
יהיה לנו ב VA שדה אינדקס לכל רמה של טבלה.

טבלת דפים מהופכת -

ממפים מסגרת מזכרון הראשי לדפים של תהליכים.
שימוש בטבלה זו חוסך כמות גדולה של זכרון ולכן ניתן להחזיק אותה ברמות גבוהות של היררכית הזכרון.

אלגוריתמים להחלפת דפים

- האלגוריתם האופטימלי

מחליף דף שההתייחסות אליו תקרה בעתיד הרחוק ביותר. לא ישים.

- Not Recently Used - NRU

מחלקים את הדפים ל-4 קבוצות: - משמע מחסנית.

- R=0, M=0 ← עדיפות גבוהה
- R=0, M=1
- R=1, M=0
- R=1, M=1

בחירת הדף תעשה באופן אקראי מתוך הקבוצה הלא ריקה בעלת העדיפות הגבוהה ביותר.

יתרונות - קל למימוש

חסרונות - 2 - דפים לא אופטימליים.
- בחירה של דפים בצורה אקראית גורמת למס' גבוה של פסיקות דף.

- תור FIFO

הדפים מוחזקים בתור מהותיק ביותר לחדש ביותר. מפנים את הותיק ביותר.

יתרונות - קל למימוש

חסרונות - מתייחסים רק לותק של דף ולא לנחיצות ולכן יכול לגרום למס' גבוה של פסיקות דף.

- הזדמנות שנייה Second Chance שכלול של FIFO

כמו FIFO אבל בוחנים גם את R של הדף הותיק ביותר.

אם R=0 מפנים את הדף, אם R=1 מעבירים את הדף לסוף התור, משנים את R=0 וממשיכים בסריקה.

* ביצועים סבירים במונחים של פסיקות דף כי מתייחסים לא רק לוותק אלא גם לנחיצות.

- אלגוריתם השעון Clock Page Replacement שכלול של הזדמנות שנייה

כמו הזדמנות שנייה רק במקום תור, מוחזקים ברשומה מקושרת מעגלית.

אם R=0 מפנים את הדף, אם R=1 מאפסים את הסיבית וממשיכים בסריקה מעגלית לדף הבא הותיק בעגול.

- Least Recently Used - LRU

מחזיקים דפים ממוינים לפי זמן התייחסות: מפנים את הדף שההתייחסו אליו בעבר הכי רחוק.

* ביצועים קרובים לאלגוריתם האופטימלי

יתרונות - יעיל

חסרונות - עלות גבוהה של החזקת הדפים ממוינים לפי זמן התייחסות. בכל פנייה לדף צריך לעדכן מיקומו ביחס לשאר.

- Aging שכלול של LRU

לכל דף מוחזק מונה שגדל ב-1 בסיבית השמאלית ביותר אם הייתה אליו פנייה בפרק זמן מסוים שנקבע.

לפני כל אינקרמנטציה של מונה עושים לו הזזה ימינה כדי לא להתייחס להיסטוריה ישנה מדי.

כשצריך להחליף דף בוחרים את בעל המונה הקטן ביותר. אם ישנם כאלה בוחרים אקראית.

- NoT Frequently Used - NFU גרסה של aging

גרסה של aging ללא הזזת המונה בכל פעם.

חסרונות - המונה יכול לצבור מידע היסטורי לא רלוונטי.

אלגוריתם המבוסס על Working Set -

קבוצת עבודה של תהליך היא קבוצת דפי הזכרון הנחוצים לריצה של התהליך בשלב הנוכחי של הריצה.
אם קבוצת עבודה של תהליך נמצאת בזכרון תהליך ירוץ בלי לגרום למס' רב של פסיקות.

אסטרטגיות הבאת דפים

* demand paging - הבאת הדפים לפי הצורך.
* prepaging - אם יודעים מהי קבוצת העבודה, אפשר להביא אותה לזכרון לפני שהתהליך מתחיל לרוץ. טכניקה זו יכולה לחסוך כמות ניכרת של פסיקות דף.

סחרור - מתרחש כאשר תהליך גורם לפסיקת דף מיד אחרי ביצוע של מס' קטן של הוראות קוד. התהליך כמעט ולא מתקדם מכיוון שמרבית הזמן מתבזבז על החלפת דפים בין הזכרון הראשי לזכרון המשני.

תיאור האלגוריתם-

אם $R=1$ - מעדכנים את הזמן הוירטואלי של הדף לזמן הנוכחי ומשנים $R=0$. מתקדמים לדף הבא.
אם $R=0$ הדף מועמד לפינוי:

1. מחשבים $age = \text{current virtual time} - \text{time of last use}$
2. אם $age > T$ הדף לא בקבוצת העבודה ולכן מפונה.
3. אם $age \leq T$ הדף בקבוצת העבודה.

עוברים על כל הדפים לפי ותיקות ומפנים את הותיק ביותר עם $R=0$.
אם לא מצאנו כזה דף, בוחרים באקראי דף עם $R=1$ ומפנים אותו. בעדיפות שיהיה עם $M=0$.

יתרונות- מספק ביצועים טובים במונחים של פסיקות דף.

WSClock-

גרסה מעגלית של האלגוריתם הקודם.
הדפים מנוהלים ברשימה מעגלית.

תיאור האלגוריתם-

אם $R=1$ - מעדכנים את הזמן הוירטואלי של הדף לזמן הנוכחי ומשנים $R=0$. מתקדמים לדף הבא.
אם $R=0$ הדף מועמד לפינוי:

1. מחשבים $age = \text{current virtual time} - \text{time of last use}$
2. אם $age > T$ הדף לא בקבוצת העבודה.
   אם $M=0$ לפנות את הדף.
   אחרת אם $M=1$, מתזמן כתיבה של הדף לדיסק וקדם את המחוג לדף הבא. תזמון לכתיבה לכל היותר n פעמים.
3. אם $age \leq T$ הדף בקבוצת העבודה.
   מבצעים מעקב אחרי הדף הותיק ביותר. (עדיפות עם $M=0$).

לאחר מעבר מעגלי כאשר עדיין לא פונה דף יתכנו שני מצבים-
* אם תוזמנה לפחות כתיבה אחת קודם, ממשיך לקדם את המחוג עד שימצא דף עם $M=0$ ופנה אותו.
* אם לא תוזמנה אף כתיבה, כל הדפים הם בקבוצת העבודה. מסמנים את הדף שעקבנו אחריו ב-3. אם אין כזה מוצאים דף עם $M=1$, כותבים אותו לדיסק ומפנים אותו.

יתרונות- מימוש יעיל של האלגוריתם הקודם, ביצועים טובים.

שיקולים מתכנון של מערכת דפדוף בזיכרון

מדיניות החלפת דפים -

מדיניות לוקלית - פינוי מתבצע רק מתוך קבוצת הדפים של התהליך הדורש דף נוסף.
מדיניות גלובלית - פינוי דפים ללא כל קשר לזהות התהליך הדורש דף נוסף.

ניהול כמות הקצאות המסגרות Page Fault Frequency - PFF

- אם יודעים אילו תהליכים גורמים במידה מתמדת למספר רב של פסיקות דף ⇐ מדיניות החלפה פשוטה בקבוצת הקבועה שלהם
- אם תהליך מתבצע כמעט בלי פסיקות דף ⇐ אולי יש לו עודף מסגרות וניתן לצמצם את גודל של קבוצת העבודה.

מנגנון אלגוריתם זה סטטיסטי. מודדים את תדירות פסיקות דף של כל תהליך ובהתאם מגדילים/מקטינים את ההקצאות שלו.
* ניתן להשתמש בו גם עבור החלפה לוקלית וגם עבור החלפה גלובלית.

קביעת גודל של דף

* דפים קטנים - פחות בזבוז בדף האחרון של כל תהליך. הריסוק הפנימי בדף האחרון בממוצע מחצית מגודל הדף.
* דפים גדולים - מקטינים את גודלה של טבלת הדפים.

s גודל של תהליך ממוצע
p גודל הדף
e גודל של שורה בטבלת הדפים

כמות דפים לתהליך $\frac{s}{p}$

גודל טבלת דפים $\frac{s}{p} \cdot e$

בזבוז עקב ריסוק פנימי בדף האחרון $\frac{p}{2}$

תקורה כללית $\frac{s}{p} \cdot e + \frac{p}{2}$

נגזור ונחפש מינימום $-\frac{s}{p^2} \cdot e + \frac{1}{2} = 0$

גודל דף $p = \sqrt{2se}$

דפים לשימוש משותף

COW - Copy On Write - טכניקה המשמשת במקרה של שיתוף דפים שמסומנים גם לכתיבה. הדפים יכולים להיות משותפים עד לניסיון הכתיבה הראשון. כאשר תהליך מנסה ניסיון כתיבה לדף כזה הוא גורם לפסיקה שבעקבותיה מערכת ההפעלה משכפלת דף וכך יתאפשר לתהליך הכותב לכתוב שינויים בעותק שלו.

יתרון - חוסך מקום בזיכרון וזמן העתקה נחסך עד שינויים.
חסרון - דפים אלה יכולים להתפנות מהזיכרון כאשר אחד התהליכים שהשתמש בהם מתפנה. ניתן לפנות אותם רק כאשר כל התהליכים שמשתמשים בהם גמרו את השימוש בהם.

שדים מדפדפים

שד מדפדף - תהליך רקע המייעל את מערכת הדפדוף ע"י כך שהוא דואג להחזיק מאגר מספיק גדול של מסגרות פנויות לשימוש. מדי פעם בודק את הכמות, אם היא קטנה מדי אז הוא בוחר דפים לפינוי בהתאם למדיניות פינוי הדפים של המערכת - cleaning policy.

סגמנטציה –
השיטה בו מרחב הזכרון המדומה מחולק לקטעים. הקטעים יכולים להיות בגדלים שונים.

מס' הסגמנט	Offset בתוך הסגמנט

* מיפוי של תהליך לסגמנטים מוחזק ב Segment Table.
* לכל סגמנט כתובת base - כתובת הפיזית של תחילת הסגמנט בזכרון הראשי.
* סגמנטים המרכיבים את התהליך לא חייבים לשכון יחד בזכרון הראשי. (יכולים להיות בזכרון המשני ובעת הפסקת מעבד יפנו לזכרון הראשי ויעודכנו בטבלת הסגמנטים).
* בשונה משיטת הדפדוף שיטה זו לא שקופה בהכרח למתכנת והיא נועדה לאפשר למתכנת להגדיר סגמנטים לפי צרכי האפליקציה.
* חלוקת הזכרון לסגמנטים בגדלים שונים יוצרת בזבוז של שטחים מבוזבזים עקב ריסוק חיצוני.

השיטה המעורבת – חלוקה לסגמנטים עם דפדוף –

- מרחב הזכרון המדומה של התהליך מחולק לקטעים לפי שיקולים של המתכנת.
- כל הסגמנטים מחולקים לדפים בגודל זהה.
- הזכרון הראשי מחולק למסגרות בגודל זהה לגודל הדפים.
- לכל סגמנט טבלת דפים המכילה מיפוי של הדפים המרכיבים סגמנט זה.

יתרונות –
השיטה המעורבת מתאפיינת בהיעדר ריסוק חיצוני (כמו שיש בשיטת הסגמנטציה).
מאפשרת שליטה של המתכנת על חלוקת מרחב הזכרון של תהליך לקטעים (לא התאפשר בשיטת הדפדוף).

מערכת קבצים

* ב Windows סיומת קובץ מגדירה את סוגו.
* ב Linux קובצי הרצה אינם צריכים לשאת סיומת מיוחדת.

מבנה פנימי של קובץ - קובץ המורכב מבתים, קובץ המורכב מרשומות וקובץ המורכב מרשומות מאורגנות במבנה לחיפוש מהיר.

* **קבצים רגילים** - יכולים להכיל מידע כלשהו לשימוש משתמש. מתחלקים לקבצי ASCII וקבצים בינאריים.
* **קבצים מיוחדים** - קבצים פנימיים של מערכת הקבצים המשמשים לארגונה.

* **Symbolic link** - סוג קובץ המשמש כהפניה לקובץ אחר. קישור סמלי אינו מונע מחיקה של קובץ היעד. אם קובץ היעד נמחק ניסיון לפנות אליו עם קישור סמלי גורר שגיאה.
* **hard link** - יצירת שם לקובץ. ניתן לקשר קובץ אחד לשתי ספריות (או יותר) שונות. אם קיים יותר מקישור אחד לקובץ, רק מחיקת כל הקישורים גורמת למחיקת הנתונים מהדיסק.

* **גישה סדרתית** - ניתן לגשת לתוכן הקובץ לפי הסדר החל מתחילת הקובץ.
* **גישה אקראית** - ניתן לגשת לתוכן הקובץ לאו דווקא על פי סדר הופעתו בקובץ.

* **מאפיינים של קבצים** - הרשאות, owner, group, זמן גישה אחרון, זמן עדכון אחרון, זמן עדכון המאפיינים האחרון, גודל הקובץ, מס' הקישורים לקובץ (מושפע ממס' ה-hard links ומפעולות פתיחת הקובץ), מזהה התקן, מזהה ההתקן לקובץ, מספר i-node.

* **נתיב מלא** - מציין את הנתיב משורש עץ הספריות אל הקובץ.
* **נתיב יחסי** - מציין את הנתיב לקובץ החל מספריית העבודה הנוכחית (הספרייה שבה נמצא המשתמש ברגע הפנייה לקובץ).

- עץ ספריות הוא גרף מכוון ללא מעגלים.
קיים רק נתיב אחד לכל היותר משורש העץ אל קובץ.
כל נתיב לקובץ שהוא נתיב יחיד ניתן לתיאור ע"י כמות אינסופית של מחרוזות נתיב.

INODES

ספריות של מערכת קבצים מכילות רשומות המורכבות מ: שם של קובץ, מס' I-node.
לפי מס' ה I-node ניתן להגיע לטבלת ה - I-node.
טבלה זו מכילה 3 סוגי שדות -
* direct - מכילים ישירות את כתובות הבלוק
* indirect - מכיל כתובת של בלוק פנימי של מערכת הקבצים המשמש לאחסון כתובות של בלוקים נוספים של אותו קובץ.
* double indirect - מכיל כתובת של בלוק פנימי, שמכיל כתובות של בלוקים פנימיים אשר מכילים את המספור של בלוקים של data.
- גישה לקבצים גדולים דורשת כמות גדולה של פניות לדיסק מכיוון שצריך להביא לזכרון בלוקים פנימיים שמכילים כתובות של בלוקים אחרים. כדי לחסוך בכמות הגישות לדיסק משתמשים גם בהטמנה.

גיבוי מערכת הקבצים

* גיבוי ברמה הפיזית - עוברים על הבלוקים של הדיסק ומגבים בלוקים שהשתנו ללא כל התייחסות של מבנה הנתונים.
* גיבוי ברמה הלוגית - מתבצע בהתייחס לשינויים שהתרחשו במערכת הקבצים.

עקביות מערכת הקבצים

מקרים של חוסר עקביות:
- בלוק חסר - לא מופיע ברשימת הבלוקים הפנויים ולא מוקצה לשום קובץ.
- בלוק חופשי מיותר - מופיע ברשימת הבלוקים הפנויים יותר מפעם אחת.
- בלוק עם הקצאה עודפת - מוקצה ליותר מקובץ אחד.
- בלוק חופשי ומוקצה - מופיע ברשימת הבלוקים הפנויים וברשימת הבלוקים המוקצים.

זכרון מטמון

מערכת ההפעלה מקצה בזכרון הראשי מסגרות המשמשות את מערכת הקבצים לצורך הטמנה של בלוקים של דיסק. מבנה הנתונים בו נעשה שימוש לצורך זה - buffer cache. כל בלוק בדיסק ממופה ע"י פונקציית ערבול לרשימת בלוקים המנוהלים ע"י LRU.
ניתן לממש כאן LRU מכיוון שבעבודה עם דיסק אין מרכיב קריטי כלשהו של זמן גישה לנתונים. יחסית לזמן הבאת בלוק, זמן התשובה של LRU אינו מהווה צוואר בקבוק.

* **Write-through** - מתבצעת כתיבה של כל העדכונים בדיסק ללא השהייה.
* **non-write-through** (UNIX) - מבצעות שמירה של עדכונים בדיסק רק אחת לפרק זמן כלשהו. פרק זמן זה קובע את היחס בין יעילות העדכון לבין מהימנותו.

מעקב אחר שטח חופשי בדיסק

- ניהול בלוקים פנויים בעזרת מפת סיביות.
- ניהול בלוקים פנויים בעזרת רשימה מקושרת של הבלוקים המכילים את רשימות הבלוקים הפנויים.

קלט/פלט

Direct Memory Access - DMA

בקר שתומך ב DMA מקבל הוראה לביצוע פעולת קלט/פלט כאשר היא כוללת את הכתובת בזכרון הראשי שבה יש לאחסן נתונים או שממנה מתבצעת העתקת נתונים. לאחר קבלת ההוראה הבקר מבצע פעולת קלט/פלט ואינו נזקק לזמן מעבד, כלומר הוא מפנה את המעבד לעשות דברים אחרים, מגדילות רמת המקביליות של המעבד. כשהוא מסיים, שולח פסיקה למעבד, מערכת ההפעלה מטפלת בה ומבצעת תזמון תהליכים לאחר תום הטיפול בפסיקה.

תכנת קלט/פלט - חלוקה ל-4 שכבות

- מנהל הפסיקות - Interrupt Handler - עוסקת בהסתרה של פסיקות חומרה.
- מתאמי התקנים Device Drivers - מכילה מתאמים להתקני I/O שונים. מתאם ההתקן הוא תוכנית שמספקת ממשק להפעלת התקן כך שכמוס לשכבה העליונה שהיא שכבה שלא תלויה בהתקן.
- תכנת I/O משוחררת מאילוצי חומרה - התפקידים של תוכנה זו:
  * קביעת ממשקים אחידים לשכבה של מתאמי התקנים.
  * הטמנה של נתוני I/O בחוצצי ביניים.
  * טיפול בשגיאות
  * הקצאה ושחרור של התקנים שאינם ניתנים לשיתוף.

בשכבה זו לרוב ממומש מנגנון הגנה על הקבצים בעזרת Capability list, access control list

- תוכנת I/O ברמת תהליכי משתמש - שדים המתאימים לארגון הפלט.

דיסקים מגנטיים

דיסק קשיח מורכב מכמה דיסקים נפרדים המסתובבים על ציר משותף בחלל ריק.
כל דיסק מחולק למסלולים tracks.
כל מסלול מחולק לגזרות Sectors.
אוסף המסלולים בכל התקליטים שנמצאים במרחק זהה מציר הסיבוב נקרא גליל או צילינדר.
המידע נשמר על שני צדי התקליטור של כל דיסק.

מערך הדיסקים - RAID

שיטה לאחסון נתונים בכמה דיסקים במקביל, מיועדת להגביר את שרידותם ונאמנותם של הנתונים ולספק קצבים גבוהים של העברת נתונים וטיפול בבקשות.

- Raid 0 - משתמשים בכל הזכרון. אין גיבוי.
- Raid 1 - חצי לגיבוי חצי לזכרון.
- Raid 2 - מפלק את המידע בכל הדיסקים. תיקון עם המינג קוד. 4 ביטים של מידע + 3 ביטים של תיקון = 7 ביטים.
- Raid 3 - כמו 2, משתמש ב- parity bit
- Raid 4 - דומה ל-0. יש גיבוי באחד הדיסקים
- Raid 5 - מפזרים את ה parity בין הדיסקים. דיסק נוסף אחד לגיבוי.
- Raid 6 - דומה ל-5. שני דיסקים נוספים.

אלגוריתמים לתזמון בראש הדיסק

- תור First Come First Served FCFS
פועל על המסלולים לפי סדר הופעת הבקשות.
יתרונות- פשוט למימוש, הוגן ביותר.
חסרונות- תזוזות רבות של ראש הדיסק.

- חיפוש הזמן הקצר Shortest Seek First- SSF
אלגוריתם חמדן. יחפש את ההפרש המינימלי
בוחר את המסלול הקרוב ביותר לראש הדיסק.
יתרונות- מקטין את מס' התזוזות של ראש הדיסק.
חסרונות- יכול לגרום הרעבה של בקשות שנמצאות במסלולים מרוחקים מהראש.

- מעלית Elevator / סריקה SCAN
עובר על הבקשות בדומה למעלית - עובר על בקשות שכיוונן מהמסלולים הפנימיים למסלולים החיצוניים ואח"כ הופך את הכיוון. מתעדף מסלולים אמצעיים.
חסרונות- מתעדף מסלולים אמצעיים, בקשות ממסלולים בפריפריה יקופחו (הרעבה).

- סריקה חוזרת C-SCAN וריאציה של המעלית.
וריאציה של אלגוריתם המעלית עם כיוון סריקה אחד בלבד. עם סיום הסריקה ראש הדיסק חוזר במהירות להתחלה וממשיך לסרוק באותו כיוון. במקום לעלות, לרדת בהתאם לדרישות הנוכחיות, המעלית עולה בה תמידה תוך מילוי הבקשות בכיוון זה.
יתרונות- אינו עובר על מסלולים אמצעיים פעמיים.
חסרונות- ייתכן שבזמן שהוא מגיע לטפל במסלול, כבר מתעדכנו רשימה ארוכה של בקשות לאותו מסלול.

ב CSCAN וב-FIFO ניתן להגביל את מס' הבקשות לאותו מסלול ולהביא לשיפור. הם הבטוחים ביותר למניעת הרעבה.

קצב העברת נתונים של דיסק שמהירות הסיבוב שלו X rpm כאשר בכל מסלול משוכים Y bytes של נתונים: $X \cdot Y \ \frac{bytes}{min}$

קיפאון

* משאבים הניתנים להפקעה - מעבד, זכרון, מצב. ניתנים לתחלוף ללא נזק!
* משאבים בלעדיים - מדפסת, קבצים, הרשומות של טבלאות פנימיות של מערכת ההפעלה.

תנאים לקיפאון -

1. מניעה הדדית - כל משאב יכול להיות מוקצה לתהליך אחד בלבד ביותר או להיות פנוי.
2. החזקה והמתנה - תהליך המחזיק משאב יכול לדרוש משאב אחר מבלי לשחרר את המשאב שהוא מחזיק בו.
3. אין תחלוף כפוי - רק התהליך המחזיק במשאב יכול לשחררו.
4. מעגל ההמתנה - כל תהליך הממתין למשאב הנדרש ע"י תהליך אחר במעגל.

1-3 תכונות הכרחיות של מערכת לקיום מצב קיפאון. 4 תנאי מספיק.
* כאשר המשאבים הנדרשים אינם משאבי מערכת אלא משאבים לוגיים התנאי היחיד להיווצרות קיפאון הוא המתנה מעגלית.

אסטרטגיות להתמודדות עם הבעיה -

1. התעלמות - שיטת בת היענה.
2. גילוי והיחלצות
3. התחמקות
4. מניעה

גילוי קיפאון והיחלצות ממנו

גילוי

* כשיש משאב יחיד מכל סוג - ניתן לחפש מעגלים בגרף הקצאת המשאבים בעזרת DFS. אם אין מעגל אין קיפאון.
* כשהמשאבים מחולקים למחלקות - מתואר אלגוריתם במדריך הלמידה.

היחלצות

- החלמה ע"י תחלוף כפוי - יכול לגרום נזק כך ולא פשוט לבחור איזה תהליך לתחלף
- היחלצות ע"י גלגול לאחור roll back -

שמירה תקופתית על הקובץ של מצבים עדכניים, כאשר מתגלה מצב קיפאון, אפשר לגלגל את התהליך שגרם לסגירת מעגל הקיפאון למצב העדכני האחרון הידוע וכך להיחלץ מהקיפאון.
יתרון - התהליך מתחיל ממצב עדכני ידוע רק מתאפשר שחזור אוטומטי.
חסרון - קיימים מצבים בהם שיטה זו יכולה להביא יותר נזק מתועלת.

- היחלצות ע"י הריגת תהליכים - פעולה אלימה. הקורבן לא חייב להיות תהליך קבוע.
יתרון - הריגת תהליך גורמת לשחרור מוחלט של משאבים.
שימושי כאשר לאחר תחלוף כפוי או גלגול לאחור מצב הקיפאון חוזר שוב אחרי תקופה קצרה מאוד.

התחמקות

מדיניות נכונה יכולה לשמור מהתהליכים מפני כניסה לקיפאון. מסובך - דורשת את ידיעת העתיד.
כל הקבוצה של תהליכים המשתפים ביניהם משאבים לוגיים תמנה לעצמה מנהל שיקצה את המשאבים בהתאם למדיניות שנקבעה.
* מסלול בטוח - מסלול העובר ממצב ההתחלה של התהליכים אל מצב שבו כולם מסיימים בלי לחצות מצב שבו ייתכן קיפאון הרי שיש בידינו סדרת הקצאות ושחרורים המאפשרת השלמת התהליכים בלי קיפאון.
* מצב בטוח - התהליכים שבו אינם קפואים ויש דרך להשלים את כל הבקשות של התהליכים כך שכולם יסתיימו בוודאות ללא קיפאון.
* מצב מסוכן - התהליכים יכולים להסתיים ללא קיפאון אך אין ערובה לכך.

אלגוריתם הבנקאים עבור משאב יחיד ועבור מחלקות משאבים

*מצב נחשב לבטוח אם קיימת אפשרות אחת לפחות של הרצה סדרתית של תהליכים (בזה אחר זה ולא במקביל) כך שכולם יסתיימו. כל עוד הרצה סדרתית קיימת, המצב נחשב לבטוח.

*מצב בטוח מבטיח שגם במקרה של דרישות מקסימליות לא ייווצר קיפאון לפחות ע"י הרצה סדרתית.

*אלגוריתם זה מתאים למערכת סגורה שבה כל התהליכים ודרכיהם ידועים מראש.
לכן מדובר בפתרון תאורטי נאה אך לא ישים.

מניעת קיפאון

- תקיפת תנאי המניעה ההדדית- הפרדת משאב בלעדי למשאב הניתן לחלוקה
ניתן לעשות זאת עם ספולרים - הם מקבלים בקשות ומעבירים רק משאב אחד יחיד.
לא תמיד קיימת אפשרות כזו - למשל הקטע הקריטי.
*צריך להפוך משאבים בלעדיים הניתנים לחלוקה למשאבים הניתנים לחלוקה.
*אין להקצות משאב אלא אם כן הוא הכרחי לתהליך.
- תקיפת תנאי ההחזקה וההמתנה- רוצים למנוע המתנה למשאבים נוספים.
בעיות שנוצרות-
- לא כל תהליך יודע את צרכיו מראש.
- הקצאה מראש של כל המשאבים יוצרת בזבוז משאבים עצום.

* דרך נוספת - לדרוש מתהליך שמבקש משאב חדש לשחרר את המשאבים שהוא מחזיק כרגע, ואז לבקש בבת אחת את כל המשאבים שהוא צריך. מתקשר לבעיית הפילוסופים הסועדים.

- תקיפת תנאי החילוף הכפוי-
שיטה אלימה ומחירה כבד.
- תקיפת תנאי ההמתנה המעגלית-
1. ע"י הקצאת משאב יחיד לתהליך. לא מעשית.
2. הקצאת משאבים בסדר קבוע מראש. לא ישימה - קשה למספר את כל המשאבים האפשריים.

הרעבה Vs. קיפאון

הרעבה- תהליך לא יכול להמשיך במהלכו התקין בגלל המתנה ארוכה מאוד למשאב אחד לפחות. מסיימת תהליך לא קשורה לפעולותיו שלו. משאב תיוני נמנע ממנו לא קשור למעשיו, אלא בעקבות מדיניות של הסביבה החיצונית.
קיפאון - לתהליך הקפוא יש חלק במצב שהביא אותו לשיתוק.

FIFO - שיטה חסינה מפני הרעבה.
*הקצאת משאבים בהתאם לעדיפויות קבועות גורמת להרעבה. ניתן לפתור זאת ע"י הוספת מנגנון aging אשר מבטיח כי תהליך הממתין זמן רב עדיפותו תגדל.

מנגנון נעילה בשני שלבים-

שיטה למניעת קיפאון שנהוגה בעיקר במערכות מסדי נתונים. תהליך שמבקש לבצע עדכונים שמערבות כמה רשומות מבקש לנעול את כל הרשומות הדרושות. אם שלב הנעילה מצליח התהליך מבצע את העדכונים ומשחרר את הרשומות. אם הנעילה של רשומה אחת לפחות נכשלה, התהליך משחרר את כל הרשומות ומתחיל את שלב הנעילה מחדש.

וירטואליזציה

פרק זה עוסק בעיקר בוירטואליזציה של מערכות מחשב.

Virtual Machine Monitor - VMM / hypervisor - שכבת תכנה המאפשרת יצירה והרצה של כמה מכונות וירטואליות VM אשר משתפות משאבי חומרה של המחשב שעליו מותקן ה VMM.

* מכונה וירטואלית VM - מערכת מחשב וירטואלית שאפשר להתקין עליה מערכת הפעלה.

VMM מספקת הפשטה הפשטה של מערכות מחשב. ה VM שרצות מבודדות זו מזו ואף שהן רצות על מחשב פיזי משותף נוצרת אשליה שכל VM פועלת על מכונה ייעודית.
זה דומה לוירטואליזציה של מעבד שבה כמה תהליכים רצים על מעבד משותף.

* וירטואליזציה עוזרת להגיע לניצולת גבוה יותר של משאבי החומרה של המערכות הללו ולספק, לחיסכון ניכר בעלויות התחזוקה.

שימושים טיפוסיים ומשתלמים כלכלית של וירטואליזציית מערכות -

* איחוד שרתים
* ניוד מערכת
* פריסה קלה של תכנות התקן וירטואליות
* ניפוי שגיאות ובדיקות
* מידור בהגנה נגד פריצה ווירוסים
* ענן - מיקור חוץ של אחסון המידע ותהליכי החישוב של ארגון למרכז חישובים חיצוני.

אפשרויות למיקום שכבת הוירטואליזציה VMM:

* וירטואליזציה ברמת ה ISA - Instruction Set Architecture
* וירטואליזציה ברמת HAL - Hardware Abstraction Layer — הפרק מתמקד ברמה זו
* וירטואליזציה ברמת מערכת ההפעלה
* וירטואליזציה ברמת הספריה
* וירטואליזציה ברמת האפליקציה

ISA - תרגום של פקודות ה ISA של המכונה הוירטואלית שבתוכנה.
הוראות המערכת האורחת מתורגמות לפקודות של המכונה הפיזית.
השיטה איטית ולא יעילה ולכן אין בה תמיכה מעשית כל כך.

HAL - משתמשת בדמיון שבין ארכיטקטורות מחשב - הארכיטקטורה שעליה רצה מערכת הפעלה של VM והארכיטקטורה של המכונה הפיזית שעליה רצה שכבת הוירטואליזציה.
בשל הדמיון בין הארכיטקטורות, רוב ההוראות של ה VM יכולות להתבצע ישירות על ה CPU הפיזי.

וירטואליזציה ברמת מערכת ההפעלה - מאפשרת יצירת קונטיינרים, ניהול משאבי החומרה והרשתות המוקצים לקונטיינר מתבצע במערכת מערכת ההפעלה. קונטיינרים משתפים מבני נתונים פנימיים של המערכת המארחת.

וירטואליזציה ברמת הספריה - תרגום קריאות מערכת לקריאות פונקציות של ספריות וירטואליזציה.

וירטואליזציה ברמת האפליקציה - מימוש של סביבה וירטואלית אשר מסוגלת לבצע אינטרפטציה של bytecode בenvironment runtime.

גורמים המשפיעים על רמת הוירטואליזציה-

* כמות המידור שהוירטואליזציה מספקת
* כמות המשאבים הנדרשת מהמכונה המארחת
* תקורה בביצועים
* סקלביליות- שיפור של ביצועים עקב הוספת משאבים
* גמישות מבחינות האפשרות לקומבינציות שונות של חומרה

* באופן כללי- ככל שהמימוש של הוירטואליזציה יותר קרוב לחומרה אזי ה VM ממודרת טוב יותר ממערכת המכונה המארחת.
מצד שני, מימוש קרוב יותר לחומרה דורש משאבים רבים יותר ומתבטא בגמישות מועטה יותר.

דרישות חומרה לוירטואליזציה

VM צריכה לפעול כמו המכונה האמיתית. משימה זו באחריות ה hypervisor וישלה 3 מימדים:
* ניהול משאבים - צריך לשלוט בצורה מלאה במשאבי המחשב.
* נאמנות למקור - ההתנהגות של תוכנה צריכה להיות זהה על VM ועל חומרה אמיתית.
* יעילות - ברוב המקרים, תוכנה צריכה לרוץ ללא התערבות מצידו של ה hypervisor.

ניהול משאבים- הוראות רגישות הרשאות hypervisor יכול לבצע ישירות. הוראות שאינן רגישות כמו ביטול פסיקות או שינוי מיפויים בטבלת הדפים יתורגמו על ידי למס' הוראות רגישות, הגון ישמור אינדיקציה לשינויים כדי שאפשר קה הפעלה שורצה מולין תקשוב ששינתה את מצב הפסיקות או את טבלת המיפויים.
לאינטרפטציה זו יש חסרון - היעילות.

נאמנות למקור-

* מצב רגיש- מצב שקריאתו בזמן שהמעבד נמצא במצב משתמש אינה גורמת ל TRAP.
* TRAP and emulate - כדי לבנות VMM יעיל מספיק, דרוש שכל ההוראות שעלולות להשפיע על פעולת נאמנה למקור של VMM יגרמו ל TRAP ויעבירו פיקוד ל VMM, והוא יעשה לכן תיקון.
לעומת זאת, פעולות לא מיוחסות יכולות להתבצע באופן ישיר ובקלות על המחשב הפיזי.
נקרא גם וירטואליזציה בתמיכה.
* VT - Virtual Technology - ה hypervisor יכול לקבוע בעזרת מפת סיביות מוגדרת אילו הוראות מכונה יגרמו ל TRAP בעת ריצתה של VM ובהמשך לבצע לביצוע התכנית בצורה שנאמנה למקור.
* תרגום בינארי hypervisor מעתיק קוד on-the-fly ומחליף הוראות רגישות בסדרת הוראות המובילה ל TRAP ל hypervisor. אחרי ה TRAP, ה hypervisor בודק את תקינות הפעולה הרגישה ומבצע אותה כאשר המעבד נמצא במצב מיוחס.
* פתרונות פרא-וירטואליזציה- מספקים סביבת וירטואליזציה דרך API של hypercalls המאפשר למערכת הפעלה אורחת להשתמש בשירותי ה VMM, לשם כך יש לשנות את מערכת ההפעלה, הקרובה בהמשך.

hypervisor מטיפוס ראשון - סוג של מערכת הפעלה, התוכנה היחידה שרצה במצב מיוחס. תפקידו להציג לפני מעלה עותקים של חומרה הנקראים VM. סוג זה של VMM נועד להריץ הן את מערכות ההפעלה הכלליות והן מערכות הפעלה ייעודיות. גודלו תלוי ביכולות ובהיקף השירותים. MB-GB.

hypervisor מטיפוס שני - תוכנה שרצה על מערכת הפעלה מארחת ומספקת שירותי VMM למערכת הפעלה אורחת או לכמה מארחות. סוג זה של VMM נועד להריץ תוכנות שנכתבו למערכת הפעלה אחת, על גבי מערכת הפעלה אחרת, גודלה יכול להיות עשרות GB.

hypervisor מטיפוס אפס - עיקר השימוש בסוג זה הוא במערכות משובצות. מדובר בשכבת תוכנה דקה מאוד של מנהל משאבים, עיקר שירותי הוירטואליזציה מסופקים ע"י חומרה, דבר זה מאפשר להקטין את היקף הקוד של ה VMM, לספק ביצועים ולהגיע לגדלי VM קטנים מאוד.

הבטים למימוש וירטואליזציה יעילה

*מצב מיוחס וירטואלי - כשמערכת ההפעלה האורחת שרצה על ה VM חושבת שהיא רצה במצב מיוחס, אך למעשה היא רצה במצב משתמש.

*מה קורה כאשר מערכת הפעלה שרצה על VM מבצעת הוראה רגישה?
- ביצוע על מעבדים ללא VT - יגרום להוראה להיכשל וייתכן אף שמערכת ההפעלה תקרוס.
- ביצוע על מעבדים עם VT - הוראה רגישה תגרום ל TRAP וה hypervisor יבחן את הקוד, אם ההוראה בוצעה ע"י מערכת ההפעלה האורחת ב VM אזי היא תבוצע ע"י ה hypervisor במצב מיוחס.
אם ההוראה בוצעה ע"י תוכנית משתמש שרצה על מערכת ההפעלה האורחת אזי ה hypervisor יבצע חיקוי של מה שהחומרה האמיתית הייתה עושה אילו קיבלה לביצוע הוראה רגישה במצב משתמש.

אופן הפעולה של תרגום בינארי/שיטת התרגום הדינאמי יחד עם חיקוי -
ה hypervisor מבצע סריקה של התוכנית לבלוקים בסיסיים, רצף הוראות ללא הוראות רגישות. בלוקים אלה מתבצעים ישירות על ה CPU. ההוראה האחרונה בבלוק הבסיסי מעבירה שליטה ל hypervisor שמחליט כיצד לחקות את הפעולה. לאחר החיקוי בוחר בלוק בסיסי חדש אשר שוב מתבצע ישירות "על ברזלים".

כדי לאפשר ל hypervisor מטיפוס 2 גישה מתאימה לביצוע הפעולות האלו בצורה נאמנה למקור רוב ה hypervisors משתמשים ברכיב שהוא כחלק של גרעין במערכת ההפעלה המארחת. כך הוראות של הרכיב מקבלות גישה לפי ההרשאות של ring 0 - מצב בו אפשר לבצע כל הוראה.

מיקרוקרנל של hypervisor

שני סוגי ה hypervisor מסוגלים להריץ מערכות הפעלה אורחות ללא שינוי, יכולת זו נקראת וירטואליזציה מלאה.

פרא-וירטואליזציה - גישה זו מחייבת שינוי במערכת ההפעלה. בשיטה זו מוחלפות כל הקריאות להוראות רגישות במערכת ההפעלה האורחת בקריאות API של ה hypervisor המכונות hypercalls.
הסרה של הוראות והחלפתן ב hypercalls הופכת את ה hypervisor למיקרוקרנל.
יתרונות - ה VMM לא נדרש לממש סמנטיקה מסובכת למדי של הוראות רגישות. יעיל יותר לקרוא ל hypercall.
חסרונות - עדיין לא הפכו שיטה זו לסטנדרט ולא תמיד ניתן להשתמש בה (למשל כשהקוד של המערכת סגור).

וירטואליזציה של זכרון

* Shadow page table - טבלה המספקת מיפוי המקשר בין מסגרות של החומרה הוירטואלית של המכונות לבין מסגרות פיזיות של חומרה אמיתית.

כיצד ידע ה hypervisor שעליו לעדכן טבלה זו?

1) טכניקה אחת היא לשמור את המיקום של טבלת דפים וירטואלית ברמה העליונה בפעם הראשונה שמנסים לטעון אותה, מאחר שכל הוראה לגישה ניתן לזהות אותה ב hypervisor.
ה hypervisor יכול למפות את טבלאות הדפים שטבלת הדפים שברמה העליונה מצביעה אליהן לדפים שהם read only.
כך כל ניסיון לערוך שינוי בטבלאות דפים יגרום לשינוי שאפשר להבחין ב hypervisor.

2) טכניקה שנייה מאפשרת למכונה מאורחת להוסיף מיפויים חדשים.
שינוי מיפויים אלה אינו גורם לשינוי ב shadow page table מכיוון שה hypervisor כלל אינו מודע להתרחשות השינוי!
כאשר מתרחשת גישה לפי המיפוי החדש, השליטה תועבר ל hypervisor והוא בוחן את טבלת הדפים של המכונה ומעדכן את המיפוי ב shadow page table.

* טכניקות אלה גורמות למס לא מבוטל של פסיקות דף ולכן הן יקרות.

* VM exit - העברת השליטה מן ה VM ל hypervisor. פעולה מורכבת ויקרה.

* nested page tables -
אנו נדרשים למפות VA ב VM אל PA ב VM
ו PA ב- VM ל PA במכונה המארחת.
ללא תמיכה של חומרה המיפוי האחרון הנדרש עם shadow page table יהיה יקר.
extended/nested page table נמצאות במעבדים מסויימים ומספקת תמיכה - ה CPU תומך בתרגום בשתי הרמות הנדרשות ללא התערבות של תוכנה.

* memory overcommitment - ה hypervisor מתחייב ל VM עם זכרון שיותר גדול מהזכרון הפיזי של מערכת ההפעלה.
זה נעשה כי שימוש בזכרון מסתמך במהלך חיה של VM וגם בגלל טכניקת deduplications המאפשרת לשתף דפים בעלי תוכן זהה.
הבעיה של התחייבות יתר היא - כיצד ה hypervisor יודע אילו מסגרות שדפים של מכונות וירטואליות ממופים אליהן אפשר לפנות כדי להעמיד אותן לרשות מכונות אחרות שדורשות יותר זכרון.
פתרונות - גישה נאיבית - כמות מיפוי נוספת של דפים והחלפות ו ballooning.

וירטואליזציה של I/O

יש לספק אשליה ל VM ע"י ה hypervisor של עבודה מול סוג מסוים של התקן I/O כאשר בפועל הנתונים מאוחסנים בהתקן I/O אחר.

I/O MMU

רכיב זה מתרגם כתובות של התקן I/O לכתובות של הזכרון הראשי.
טבלת מיפוי בין התקן I/O לזכרון מוחזקת ע"י hypervisor שיודע לנהל את הזכרון הראשי עבור VM.

נדרשת גם וירטואליזציה של פסיקות - פסיקות I/O אמורות להגיע ל VM הנכונה.
I/O MMU תומך בהוספת רמת מיפוי פסיקות חדשה להתקני I/O - Interrupt re mapping.

* התקני התקן וירטואלי - VMs מוכנות המכילות מערכות הפעלה יחד התוכנות הנדרשות. מאפשרות הפצה קלה של אפליקציות על גבי אמצעי אחסון ניידים ואינטרנט.

* deduplication - הרעיון מאחורי מניעת הכפילויות הוא להחזיק hashing של דפי זכרון ולזהות דפים בעלי אותו ערך, אם גם תוכן הדפים זהה ממפים לאותו תוכן משני דפים.
טכניקה זו מצריכה שימוש ב COW כדי להתמודד עם שינויים אפשריים.

ענן

* ציבורי - ספק השרות מספק את שירותי הענן דרך האינטרנט.
* פרטי - שירותי חישוב ואחסון ניתנים במרכז מחשוב פרטי.
* היברידי - ענן פרטי המספק את דרישות האחסון והחישוב של משתמשיו ומסוגל לספק שירותים מענן ציבורי בזמן העומס.

* איזו תכונה VT מוסיפה למעבד ואיך זה מספר את קידומי הוירטואליזציה?

VT מוסיפה את התכונה שמאפשרת להריץ קוד של VM ישירות על המעבד ללא כל שינויי מצב ה VM/VT. המשמעות הוא שהפעולות הללא מיוחסות יכולות להתבצע באופן ישיר ובפעולות על המעבד הפיזי. שיטה זו נקראת Emulate and Trap (עמ' 243 מדריך הלמידה) אושיטת וירטואליזציה בחומרה.

הרעיון הבסיסי הוא ש hypervisor יכול לקבוע בצורה מפת סיביות חומרתית אילו הוראות מכונה יגרמו ל Trap במהלך ריצתה של מכונה וירטואלית ובהמשך לדווג לביצוע תכנית בצורה "נאמנה למקור".

* האם במערכת הפעלה שתומכת בקונטיינרים ניתן להריץ בקונטיינרים שונים אפליקציות שונות שנבנו בשביל מערכות הפעלה בעלות Kernel שונה בלי להשתמש בוירטואליזציה?

כן, בעיקר להריץ בקונטיינרים שונים אפליקציות שנבנו בשביל מערכות הפעלה בעלות גרעין שונה אלא אם נשתמש בוירטואליזציה של חומרה.
אם לא משתמשים בוירטואליזציה שכזו, הקונטיינרים יתמכו באפליקציות שנבנו בשביל מערכות הפעלה עם אותו גרעין כמו שהקונטיינרים רצים עליו.

* יש צורך להפעיל מערכת הפעלה כלשהי כמערכת הפעלה וירטואלית מעל hypervisor שתומך ב- פרה-וירטואליזציה בלבד. הסבירו אילו חלקים של מערכת ההפעלה האורחת צריך לשנות.

יש צורך שמערכת ההפעלה האורחת תשנה את כל הקריאות הרגישות של מערכת ההפעלה הוירטואלית להיפר-קריאות ואלו יתבצעו לאחר מכן בקטרוה במצב מיוחס ע"י TRAP על גבי hypervisor.

מנגנוני הגנה

* שמירת מטריצת תחומי הגנה כרוכה בבזבוז רב מכיוון שצריך לשמור את ההרשאות של כל העצמים במערכת ועבור כל התחומים שבה.

- Access Control List - ACL - לכל קבוצת משאבים יש רשימה של קבוצות/יוזרים שיכולים לגשת אליהם ואילו רמות הרשאות יש להם.
- רשימות יכולות C-Lists - כל תהליך בתחום יחזיק מצביע לרשימה של כל המשאבים שיש לו הרשאות אליהם. בכל פעם שתהליך מבקש לבצע פעולה מסוימת רשימה זו נבדקת והפעולה מתאפשרת אמ"ם העצם אכן מופיע ברשימה זו והפעולה הנדרשת מותרת.

יתרונות וחסרונות של ACL לעומת C-Lists -

יתרונות - יעיל יותר מכיוון שאין צורך לעבור על רשימת כל המשאבים של משתמש על מנת לבדוק קובץ מסוים.

חסרונות - במידה ומוחקים משתמש יש לעדכן את כל הקבוצות של המשאבים לעומת עדכון רשימה אחת ב C-Lists ששייכת למשתמש הזה.

XV6

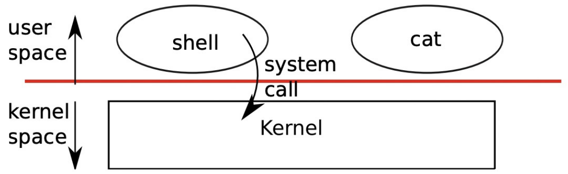


Figure 0-1. A kernel and two user processes.

*תהליך ב XV6 מכיל - הוראות, data ומחסניות.

* XV6 שומר בזכרון תהליך שלא רץ כרגע גם כל המידע הנחוץ כמו רגיסטרים ומשחזר את מידע זה כשהתהליך מוכן לרוץ.

Fork() -

* מאפשר לתהליך לייצר עותק שלו בתהליך בן - child process, לתהליך הבן מועתק גם כל הזכרון של האב ולכל אחד יש את העותק שלו. שינוי בזכרון של אחד מהם לא משפיע על האחר.
* הקריאה מחזירה PID של תהליך. 0 לתהליך הבן ומס' גדול מ- 0 לתהליך האבא.

```
1)  int pid = fork();
2)  if(pid > 0){                                  → האבא
3)    printf("parent: child=%d\n", pid);
4)    pid = wait();
5)    printf("child %d is done\n", pid);
6)  } else if(pid == 0){                          → הבן
7)    printf("child: exiting\n");
8)    exit();
9)  } else {
10)   printf("fork error\n");
11) }
```

Exit() - מפסיקה את ריצת התהליך הקודם ומשחררת את כל המשאבים שתפס.

Wait() - מחזירה את ה PID של הבן שסיים את ריצתו.
אם אף ילד לא סיימו את ריצתו, wait ממתינה עד שבן כלשהו יסיים.

* שורה 3 ושורה 7 יכולות להיות מודפסות בכל סדר.
שורה 5 תודפס רק לאחר שהבן יסיים את ריצתו ו-wait יסיים לחכות לו.

Exec() - מחליפה זכרון של התהליך הקודם. לקובץ החדש יש פורמט ספציפי: ELF.

```
char *argv[3];

argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);
printf("exec error\n");
```

1 שם הקובץ החדש
2 מערך של Strings

read(fd, buf, n) — מחזירה כמה בתים נקראו

- fd — מצביע לקובץ
- buf — הבתים שנקראו מוחזקים לכאן
- n — כמות בתים לקריאה

write(fd, buf, n) — מחזירה כמה בתים נכתבו

- fd — מצביע לקובץ
- buf — בתים לכתוב לקובץ
- n — כמה בתים לכתוב

Close() משחררת קובץ כך שניתן להשתמש בו שוב.

* XV6 ממומשת כמערכת הפעלה מונוליטית
* לכל תהליך מרחב כתובות משלו כך שלתהליכים אחרים אין גישה אליו ולו אין לשלהם.
* XV6 משתמשת בטבלאות דפים כדי למפות VA ל PA עבור כל תהליך.

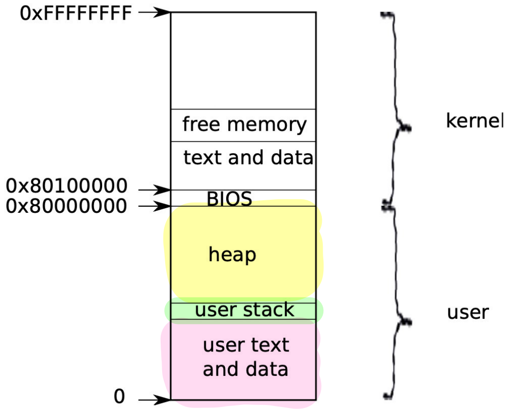


Figure 1-2. Layout of a virtual address space

* ב proc struct שומרים לכל תהליך: טבלת דפים, מחסנית שלו ב kernel ומצב הריצה שלו.
* לכל תהליך יש שתי מחסניות: kernel stack, user stack.
כשהתהליך רץ הוא משתמש ב user stack אך כשיש פסיקה והשליטה עוברת למצב מיוחס אזי הוא עובר להשתמש ב kernel stack, המידע ב user stack נשמר אך לא משתמשים בו.

* ריצה של תהליכים: המתזמן מתפס תהליך p→state = RUNNABLE
כשהוא מריץ אותו אזי המצב שלו משתנה p→state = RUNNING

* swtch נקרא כשרוצים לבצע CSW.

* ישנם 3 מקרים בהם חייבים להעביר את השליטה ממצב משתמש למצב גרעיני -
1. System call קריאת מערכת
2. exception שגיאה כאשר התכנית מבצעת פעולה לא חוקית.
3. interrupt פסיקה כאשר מכשיר שולח סיגנל על מנת לסיים שהוא צריך לתשומת לב מה OS.

מערכת הקבצים ד XV6

מטרתה לארגן ולאחסן את המידע.

מורכבת מ7 שכבות.

שכבה	
File descriptor	
Pathname	
Directory	
Inode	← מכיל מידע על בלוקים
Logging	← גבוי במטרה לסייע לשחזר אחרי קריסות
Buffer cache	← מנהלת גישה לבלוקים בדיסק, שומרת גישה מהירה לבלוקים פופולריים.
Disk	← קוראת וכותבת בלוקים על כונן קשיח.

XV6 Containers, namespaces & cgroups

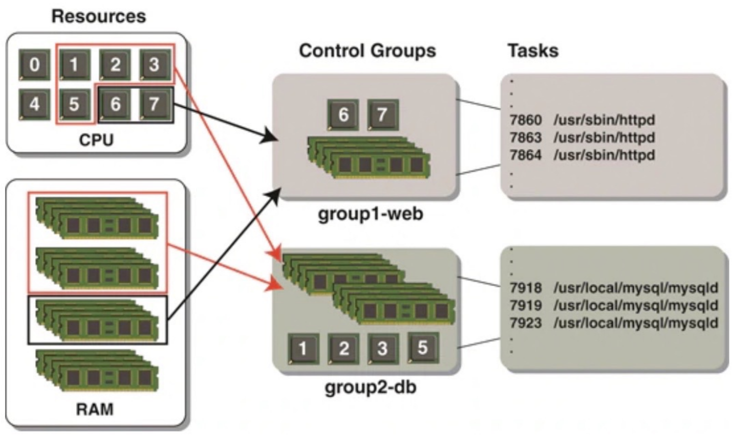
קונטיינר - סט של לפחות תהליך אחד או יותר המבודדים משאר המערכת. ניהול המשאבים מסופק ע"י קבוצות בקרה ובידוד משאבים באמצעות namespaces.

סוגים של namespace:

* PID - מספק לתהליכים קבוצה עצמאית של PID המופרדים מ namespace אחרים. תהליכים מתוך מרחב PID של הילד נגישים גם ממרחב ה PID של האב.
* Mount - מבודד ושולט בנקודות mount. מאפשרת לתהליך לקבל קבוצה משלו של מערכת קבצים מותקנות הנפרדת ובלתי תלויות מאלה שנראות ע"י תהליכים אחרים במערכת.
* Network
* UTS
* IPC
* User
* Time

Control groups - Cgroup

פיצ'ר בלינוקס המאפשר למנהל מערכת ההפעלה להקצות משאבי מערכת כגון CPU, זכרון, דיסק, I/O ועוד לקבוצות של תהליכים או משימות. הוא מספק דרך להגביל, לתעדף ולנטר את משאבי המערכת המשמשים קבוצה של תהליכים מה שיכול להיות שימושי לשיפור ביצועי המערכת.



Crgoups מיועדות ביצירת קונטיינרים.
קונטיינרים שייכים לסוג וירטואליזציה המכונה "וירטואליזציה ברמת המערכת"/"וירטואליזציה מסוג C".
וירטואליזציה זו קלת משקל מבחינת המשאבים אך מקריבה מספר היבטי אבטחה בתוך הפשרה.

מימוש קונטיינרים ב XV6

בXV6 תומכים רק ב PID וב- Mount.

pouch - פקודה זו מאפשרת ליוזר לבצע פעולות על קונטיינרים - ליצור, למחוק, לצאת...
* עד 3 קונטיינרים ניתנים ליצור.
* אין קינון של קונטיינרים

Attached mode: ‘pouch’ commands available inside a container	Detached mode: ‘pouch’ commands available outside a container
	pouch start {name}
pouch disconnect	pouch connect {name}
pouch info	pouch info {name} מידע על הקונטיינר ומצבו
	pouch list all רשימה עם מידע על כל הקונטיינרים הרצים
	pouch destroy {name}
	pouch cgroup {name} {state-object} [value]
pouch --help	pouch --help

Tab 1. ‘pouch’ commands according to the mode

deattach user terminal from a running container ← (pouch disconnect)

→ creates & starts a container (pouch start)

→ attach user terminal to a running container using his id string (pouch connect)

→ stops & destroys a running container (pouch destroy)

Name:
pouch cgroup - limit, account or control resources associated with a container that is specified with an identification string

Synopsis:
pouch cgroup *{ name } { state-object }* [*value*]
{ name } - a container identification string
{ state-object } - specified the state object name.
[value] - specify the value to assign to the state object. Note: xv6 shell doesn’t treat a string with spaces enclosed by quotes as a single argument. Thus, multiple values have to be separated using commas (see examples below).

שם של הקונטיינר ↑ מגבלה ↗ ערך ↗

Examples:
“pouch cgroup c1 cpu.max 10000” - updates cpu.max property to 10000, leaving the period default.
“pouch cgroup c1 cpu.max 10000,20000” - updates cpu.max property to 10000 and sets period to 20000.

Cgroup in xv6

* קבצים עם קידומת cgroup.xx שייכים לליבה של מנגנון ה cgroup האחראי על ארגון ההיררכיה.
* cpu.xx קבצים של קונטרולר מסוים.
* memory.xx

הסבירו למה במערכת 6XV שתומכת בקונטיינרים בעקבות כתיבת "cpu+" לקובץ
cgroup.subtree_control בבדיקה cat cgroup.subtree_control מופיע רק "cpu" ללא ה + ?

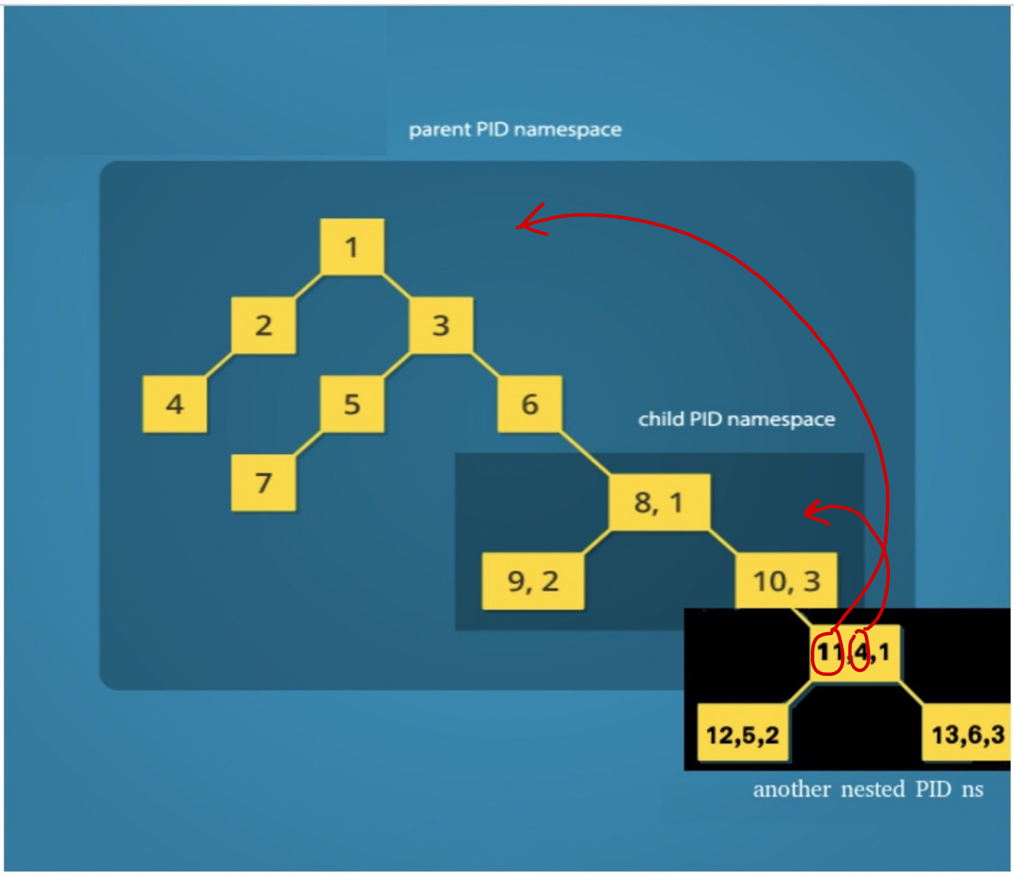
מערכת הקבצים של cgroup היא pseudo file system -הקבצים במערכת אינם קבצים "רגילים" שאליהם ניתן לכתוב מחרוזות, אלא קבצים מדומים שקריאה וכתיבה מהם נועדה למימוש פנימי של cgroup ב6xv.

cgroup.subtree_control	A read-write file that exists in non-root cgroups. The file is empty when cgroup fs is mounted. A controller name prefixed with '+' or '-' can be written to the file in order to enable/disable controllers. When read, it shows a space separated list of the controllers which are enabled to the cgroup and all it’s descendants.	Inside non-root cgroups

* cgroup יוצר מבנה היררכי וכל תהליך שייך רק לקבוצת cgroup אחת בלבד. כברירה, התהליך משתייך לאותה קבוצה שאביו שייך אליה והוא יכול לעבור לקבוצה אחרת.
* cgroup מקוננים נוצרים ע"י mkdir

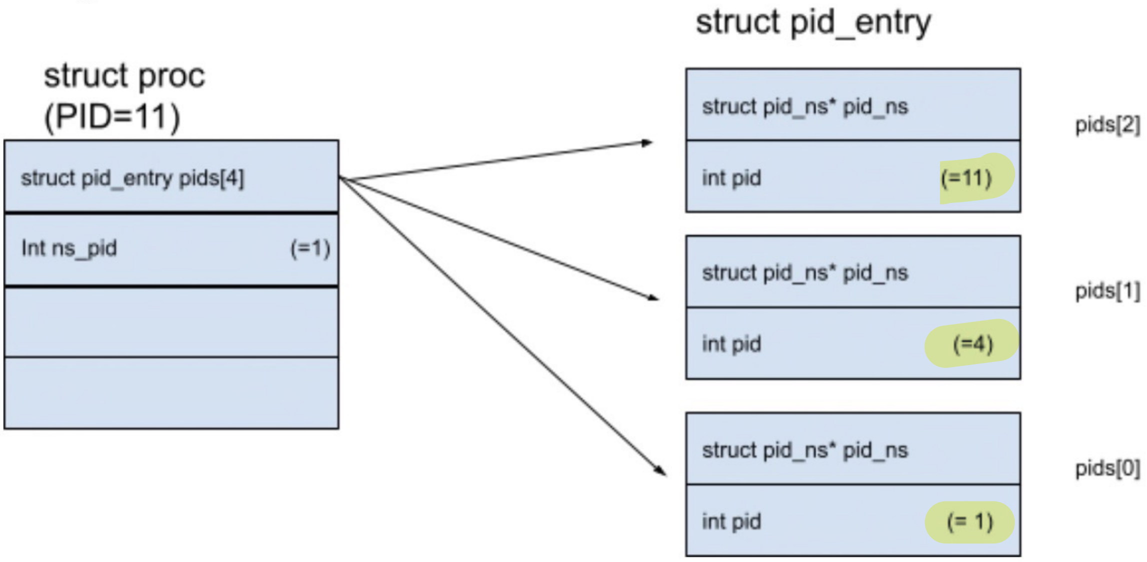
PID namespaces

* PID של הבנים גלוים משל האבא אך ההופך לא נכון.
* אם התהליך עם PID=1 מת, כל שאר התהליכים ב namespace הזה ינוקו.



Pic 16. Another level of a nested namesapce (reiterated based on Pic1)

As one can observe, process IDs in the third namespace start from PID=1. However, for the namespace at the second level it is known as a PID=4, while in the parent PID namespace it holds PID= 11. Pic. 16 describes how the array *pids[4]* of *struct pid_entry* is holding the numbers.



Pic 16. How *struct pid_entry pids* is used to implement nested PID namspaces system call

```c
int
cps108() {
  struct proc *p;
  sti();
  acquire(&ptable.lock);

  cprintf("name \t pid \t state \t \t extpid \t ppid \t cputime \n");

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {

   unsigned int cpu_time = p->cpu_time;
   int pid = p->ns_pid;
   int ppid = pid == 1 ? 0 : p->parent->ns_pid;

   int extpid = p->pids[0].pid;
   // We don't have nested containers so there will be one container at most
   if(p->pids[1].pid != 0) {
     extpid = p->pids[1].pid;
     ppid = p->parent->pids[0].pid;
   }

   if (p->state == SLEEPING || p->state == RUNNABLE || p->state == ZOMBIE) {
     cprintf("%s \t %d \t SLEEPING \t %d \t \t %d \t %d \n", p->name, pid, extpid, ppid, cpu_time);
   } else if (p->state == RUNNING) {
     cprintf("%s \t %d \t RUNNING \t %d \t \t %d \t %d \n", p->name, pid, extpid, ppid, cpu_time);
   }
  }

  release(&ptable.lock);
  return 33; //cps syscall number
}
```

memory.max

Configuration available in a non-root cgroup. After cgroup memory controller was enabled, “memory.max” will be available. This limit value stores an integer number which represents the maximum number of bytes that processes under this parent cgroup and all its children can get.
When a process moves to a cgroup with memory maximum definition and this process memory size is greater than memory.max, throw an error and do not proceed.
When a process that is inside a cgroup with memory maximum definition is being forked, check if a summary together with this new process memory size grows over the memory.max. Then it throws an error and aborts fork operation.
When a process tries to grow its memory, test for memory maximum configuration. When the new memory size of the process grows over memory.max abort with an error.
Value of -1 will remove the limit of memory.max.

memory.min

Configuration available in a non-root cgroup. After cgroup memory controller was enabled, “memory.min” will be available. This limit value stores an integer number which represents the minimum number of bytes that processes under this parent cgroup and all its children can get.
When adding a new process with smaller size than memory.min, grow this new process memory size to match the memory minimum number of bytes.
When a process is being killed, check if the memory size of a cgroup processes and its children processes is lower than defined in memory.min. When it is lower, calculate the difference and appen this difference divided by the number of all processes in cgroup parent and its children.

memory.failcnt

Configuration available in a non-root cgroup. After cgroup memory controller was enabled, “memory.failcnt” will be available. This counter increments every time a process tries to allocate more memory than the memory.max limit.

ממ״ן 11

Adding a System Call to XV6

Syscall.h → Assigns a number to each syscall

```
#define SYS_getyear 22
```

Syscall.c → Contains an array of function pointers which uses the defined numbers from syscall.h as pointers to system calls which are defined in a different location.

```
extern int sys_getyear(void);
```

function prototype נממש אותה בקובץ אחר

```
[SYS_getyear] sys_getyear,
```

מיפוי מס׳ הקריאה לפונקציה שמבצעת הפעולה שצריכה לבצע

Sysproc.c → Defenitions of system calls.

```
// return the year of which the Unix version 6 was released

int
sys_getyear(void)
{
return 1975;
}
```

מימשו כאן את פעולת הפונקציה.
* בממ״ן 11 ו-12 שלנו אנחנו מימשנו את הפונקציה בקובץ proc.c
שם ביצענו את ההדפסות של קריאת המערכת ps

Usys.S → לאחר שהקריאה מומשה יש להוסיף אותה אל ממשק המשתמש קריאות syscall שהיוזר יכול לבצע

```
SYSCALL(getyear)
```

User.h → הגדרה של קריאות המערכת (באיזה פורמט יש לקרוא להן)

```
int getyear(void);
```

לבסוף דוגמה לקריאה ע״י יוזר מתוך תכנית main():

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
printf(1, "Unix V6 was released in the year %d\n", getyear());
exit();
}
```

תר״ן 12

מימוש קונטיינרים עם PID namespace

proc.h

לא רץ → SLEEPING, RUNNABLE, ZOMBIE

רץ → RUNNING

```c
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

struct pid_entry {
  struct pid_ns* pid_ns;
  int pid;
};


// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  /* int pid;                     // Process ID */
  int ns_pid;
  struct pid_entry pids[4];
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  struct mount *cwdmount;      // Mount in which current directory lies
  char name[16];               // Process name (debugging)
  struct nsproxy *nsproxy;     // Namespace proxy object
  struct pid_ns *child_pid_ns; // PID namespace for child procs
  int status;                  // Process exit status
  char cwdp[MAX_PATH_LENGTH];  // Current directory path.
  struct cgroup * cgroup;      // The process control group.
  unsigned int cpu_time;       // Process cpu time.
  unsigned int cpu_period_time;// Cpu time in microseconds in the last accounting frame.
  unsigned int cpu_percent;   // Cpu usage percentage in the last accounting frame.
  unsigned int cpu_account_frame; // The cpu account frame.
};
```

`int ns_pid;` — PID של תהליך מקונטיינר

`struct pid_entry pids[4];` → נשתמש במערך זה כדי למצוא extpid

proc.c

```c
int
cps108() {
  struct proc *p;
  sti();
  acquire(&ptable.lock);

  cprintf("name \t pid \t state \t \t extpid \t ppid \t cputime \n");

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {

    unsigned int cpu_time = p->cpu_time;
    int pid = p->ns_pid;
    int ppid = pid == 1 ? 0 : p->parent->ns_pid;

    int extpid = p->pids[0].pid;
    // We don't have nested containers so there will be one container at most
    if(p->pids[1].pid != 0) {
      extpid = p->pids[1].pid;
      ppid = p->parent->pids[0].pid;
    }

    if (p->state == SLEEPING || p->state == RUNNABLE || p->state == ZOMBIE) {
      cprintf("%s \t %d \t SLEEPING \t %d \t \t %d \t %d \n", p->name, pid, extpid, ppid, cpu_time);
    } else if (p->state == RUNNING) {
      cprintf("%s \t %d \t RUNNING \t %d \t \t %d \t %d \n", p->name, pid, extpid, ppid, cpu_time);
    }
  }

  release(&ptable.lock);
  return 33; //cps syscall number
}
```

`int pid = p->ns_pid;` — PID של תהליך

`int ppid = ...;` — PID של אבא שלו

`if(p->pids[1].pid != 0)` → משמע הוא בן של מישהו.
צריך לבדוק מה הPID שלו מהסתכלות של האבא

```
user@ubuntu: ~/xv6
init: starting sh
$ ps
main-loop: WARNING: I/O thread spun for 1000 iterations
name     pid     state        extpid       ppid    cputime
init     1       SLEEPING     1            0       55965
sh       2       SLEEPING     2            1       15027
ps       3       RUNNING      3            2       6978
$ pouch start c1
Pouch: c1 starting
$ ps
name     pid     state        extpid       ppid    cputime
init     1       SLEEPING     1            0       55965
sh       2       SLEEPING     2            1       26633
ps       7       RUNNING      7            2       27202
pouch    5       SLEEPING     5            1       12803
sh       1       RUNNING      6            5       3771386
$ pouch connect c1

tty0 connected
ps
name     pid     state        extpid       ppid    cputime
init     1       SLEEPING     1            0       55965
sh       2       RUNNING      2            1       5162115
ps       2       RUNNING      9            1       6639
pouch    5       SLEEPING     5            1       12803
sh       1       SLEEPING     6            5       25363971
$
```

Sh

DESCRIPTION top

The *sh* utility is a command language interpreter that shall execute commands read from a command line string, the standard input, or a specified file. The application shall ensure that the commands to be executed are expressed in the language described in *Chapter 2, Shell Command Language*.

כלל | 13 מימוש קונטיינרים עם mount namespace

תהליכים/ משימות שונות אשר שייכים ל mount שונים יראו היררכיות ניתובים שונות.

Mount namespaces facilitate an isolation of mount points. In order to achieve a mountpoint segregation the *unshare* system call with MOUNT_NS parameter must be called by the process that is inclined to have a separate (hidden from other processes) view on mount points. Descendants of that process are going to inherit that separated view preserving a mountpoint segregation.

```
// ********************************************************************************
// create a child process with a separate mount namespace
// create a mount point and mount on it a preformatted internal_fs_a
// create a file on the mounted file system

int pid = fork();
if (pid < 0){
  printf(stderr, fork failed\n");
  exit(1);
 }

if (pid == 0){
   if(unshare(MOUNT_NS) != 0){
     printf(stderr, "Cannot create mount namespace\n");
     exit(1);
   }

   if (mkdir("dirA") != 0){
     printf(stderr, mkdir failed\n");
     exit(1);
   }

   if (mount("internal_fs_a","dirA",0) != 0){
     printf(stderr, "mount failed\n");
     exit(1);
   }

   createNwrite("dirA/file.txt", "123456789\n",10);
}
// ********************************************************************************
```

הבן

if(unshare(MOUNT_NS) != 0) ← מנסה ליצור לו mount חדש

mkdir("dirA") ← מנסה ליצור תיקייה חדשה

mount("internal_fs_a","dirA",0) ← מנסים להוסיף את התיקייה החדשה ל mount

createNwrite ← מייצר קובץ חדש וכותב אליו בתוך ה mount החדש

When creating a new mount namespace, the `internal_fs_a` flag can be used to specify that the new namespace should be created with an empty filesystem tree. This means that the new namespace will start with no filesystem mounts and an empty root directory.

Here is an example of how this flag can be used in the `unshare` command to create a new mount namespace with an empty filesystem tree:

css Copy code

```css
unshare --mount --internal_fs_a /bin/bash
```

```
// *****************************************************************************
if (pid > 0){
   // make sure child process runs first to create a new ns
   sleep(10000);

   // create a mount point and mount on it a preformatted internal_fs_b
   if (mkdir("dirB") != 0){
     printf(stderr, mkdir failed\n");
     exit(1);
   }

   if (mount("internal_fs_b","dirB",0) != 0){
     printf(stderr, "mount failed\n");
     exit(1);
   }

    createNwrite("dirB/file.txt", "987654321\n",10);
}


// *****************************************************************************
// both processes will sleep for a while to enable each
// other to reach this point
sleep(10000);

// *****************************************************************************
// at this point it is guaranteed that the child process is able to access
// dirA/file.txt while the parent process is able to access dirB/file.txt but
// not vice versa. We just need to check it.
if(pid == 0){
   if (open("dirA/file.txt",O_RDONLY) < 0){
     printf(stderr, "open was about to succeed but failed\n");
     exit(1);
   }
   if (open("dirB/file.txt",O_RDONLY) >= 0){
     printf(stderr, "open was about to fail but succeeded\n");
     exit(1);
   }
}
else{
   if (open("dirB/file.txt",O_RDONLY) < 0){
     printf(stderr, "open was about to succeed but failed\n");
     exit(1);
   }
   if (open("dirA/file.txt",O_RDONLY) >= 0){
     printf(stderr, "open was about to fail but succeeded\n");
     exit(1);
   }
}

exit(0);
```

Compiling and running the code snippet from above will create a mount namespace for a child process by unsharing it from the global mount namespace. Upon the namespace creation, the child process mounts the internal_fs_a, a device with a preformatted file system on it, on the dirA mountpoint. The parent process mounts the internal_fs_b device on dirB respectively.

At this stage the root directory contains dirA and dirB subfolders. But only the child process is able to see the file.txt that was created on the internal_fs_a filesystem, while only the parent process is able to access the file.txt that was created on the internal_fs_b.

After the call to unshare we would have:

Global mount namespace		New mount namespace	
Mounted device	Mount point	Mounted device	Mount point
internal_fs_b	dirB	internal_fs_a	dirA
The root file system contains an empty dirA		The root file system contains an empty dirB	

Tab 4. Mount namespaces provide different "views"

Mount namespaces facilitate an isolation of mount points. New mount namespace is created by the *unshare(MOUNT_NS)* system call. The calling process has a new mount namespace for its children which is not shared with any previously existing process. To handle mount namespaces the pseudocode of the *unshare* function described in Tab 8. is amended with the following step:

- Create a new mount namespace (*copymount_ns* function) and update the myproc()->nsproxy->mount_ns field to ensure the calling process has a segregated view on mountpoints.

Tab. 9 mount ns amendment to the unshare system call

The *copymount_ns* function reserves a row in a *mountnstable* (mount_ns.c). Actually all mount_ns structs are preallocated and as Pic 15 depicts, nsproxy[i] simply holds a pointer to the specific row in a global *mountnstable*.

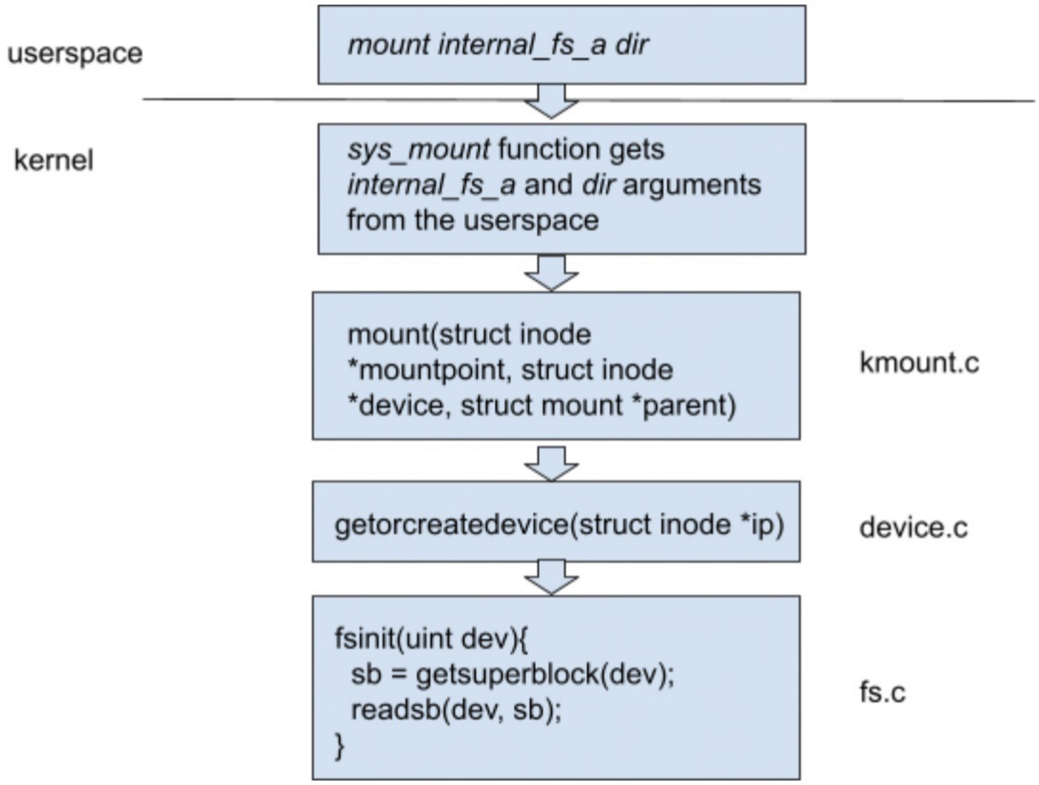
Mountpoint related structures in xv6

Mount namesapces in xv6 are preallocated and accessible via the global *mountnstable:*

```
#define NNAMESPACE   20  // maximum number of namespaces

struct {
  struct spinlock lock;
  struct mount_ns mount_ns[NNAMESPACE];
} mountnstable;
```

```
static struct mount_ns* allocmount_ns()
{
  acquire(&mountnstable.lock);
  // FIX ME  allocate proper entry to preserve a correct mountnamepaces structure

  // We need to check if there is an available namespace
   for (int i = 0; i < NNAMESPACE; i++) {
     if (mountnstable.mount_ns[i].ref == 0) {
       struct mount_ns* mount_ns = &mountnstable.mount_ns[i];
       mount_ns->ref = 1;

       release(&mountnstable.lock);
       return mount_ns;
     }
   }

   // We were unable to allocate
   release(&mountnstable.lock);
   panic("out of mount_ns objects");
}

struct mount_ns* copymount_ns()
{
  struct mount_ns* mount_ns = allocmount_ns();
  mount_ns->active_mounts = copyactivemounts();
  mount_ns->root = getroot(mount_ns->active_mounts);
  return mount_ns;
}
```

The following diagram depicts what happens in the xv6 kernel when the command line *mount* utility call is issued on a preformatted file system.



Pic 17. Mount system call trace[2]

מושגים נוספים

* **מערכת אצווה** - מערכת שיכולה לבצע באופן רצוף סדרה של משימות שהוכנה מראש. במערכת כזו כל משימה מתבצעת מתחילתה ועד סופה מבלי לשתף את המעבד בתהליכים אחרים.

* **Admission Scheduler במערכות אצווה** - ברמה זו מתבצעות החלטות תזמון "גסות". תזמון ברמה זו מבוצע לעיתים. תזמון לטווח ארוך. כיוון שהעבודות צריכות להצהיר על משאבי המערכת הדרושים להן, ניתן לתכנן סדר פעולות ארוך טווח שידע לנצל את משאבי המערכת ולהחליט אם לקבל עוד עבודות.

* **Memory Scheduler במערכת אינטרקטיביות** - תזמון לטווח בינוני. התזמון ברמה זו מעביר תהליכים ממצב מוכן בזכרון למוכן בדיסק או להפך ומרדים בזכרון לרדום בדיסק (ולהפך). ההחלטות ברמה זו נועדו לשפר את ניצולת מרכיבי החומרה.

* **CPU scheduler במערכת אינטרקטיביות** - תזמון לטווח קצר. תזמון ברמה זו מעביר מצב מוכן לרוץ והפוך. המתזמן מממש את מדיניות התזמון.

* **Priority Inversion Problem** - בעיית זיהוי מצבים שמגדירים היפוך עדיפויות. ע"מ 55 במדריך למידה.
מצב בו תהליך שעדיפותו גבוהה יותר לא מצליח להתקדם ותקוע כיוון שתהליך אחר מונע זאת ממנו. פתרון בעיה מסוג זה הוא להפוך את העדיפויות של התהליכים לזמן קצר או שימוש במבנה נתונים הניתנים לנו חסימה.

* **הסבר כשיש RAM אינסופי עדיין כדאי להשתמש במנגנון זכרון וירטואלי?** כן.
- מרחב כתובות גדול יותר מאפשר למשתמש להתייחס לתכנה כאילו היא שמורה באופן רצוף.
- תוכניות רבות יכולות לרוץ במקביל מבלי שיגמר מקום בזכרון.
- זכרון וירטואלי מספק מידה של הגנה בין התוכניות. לכל תוכנה יש את מרחב הכתובות שלה מה שמאפשר בידוד בין תוכניות (שלא יגשו לזכרון שלא מוקצה להם).
- מקטין ריסוק.

* הכניסות ב TLB הוא **אסוציאטיבי מלא** ולכן מחפשים **במקביל על כל השורות בטבלה**.
* הכניסות בזכרון מטמון **מסוג מיפוי** ישיר direct map מתבצע בכל פעם על **שורה אחת**.

* דרוג אלגוריתמים לפינוי דפים לפי ביצועים במובן של יכולת מימוש בסדר יורד:
אופטימלי > WSClock > LRU > הזדמנות שנייה > FIFO

* **זיהוי trashing סחרור** - ניתן להשתמש במונה של מס' פסיקות דף אל מול התקדמות התהליך עצמו. כשהמונה של פסיקות הדף גדל משמעותית ממונה של התקדמות התהליך ניתן לדעת כי המערכת במצב של סחרור.

* **יתרונות בשימוש ב exclusive cache** - (כל בלוק מידע מאוחסן רק בזכרון cache אחד)
1. חסכון במקום כי המידע לא נשמר במס' רמות.
2. הקטנת הסיכוי לחוסר עקביות כי לא צריך לתחזק את המידע בכמה עותקים.
3. שליפה מהירה יותר של המידע כי לא צריך לחפש בכמה רמות.
4. פשוט למימוש.

* **יתרונות בשימוש ב Inclusive cache** - (כל בלוק מידע שמאוחסן ברמה נמוכה מאוחסן גם ברמה גבוהה)
1. ניתן לגשת למידע במהירות כי הוא נמצא בשתי הרמות.
2. קל למימוש.

* כשכל השורות ב TLB מנוצלות ניתן להשתמש במנגנון LRU לפינוי שורות בטבלה - לפי מדיניות זו, השורה המוחלפת תהיה זו שלא השתמשו בה בפרק הזמן הארוך ביותר.

* אם unshare לפני ה fork, כולם באותו MOUNT_NS

* mount לפני ה unshare שנעשה האבא - גם לאבא וגם לבן יהיה גישה לקבצים ב mount